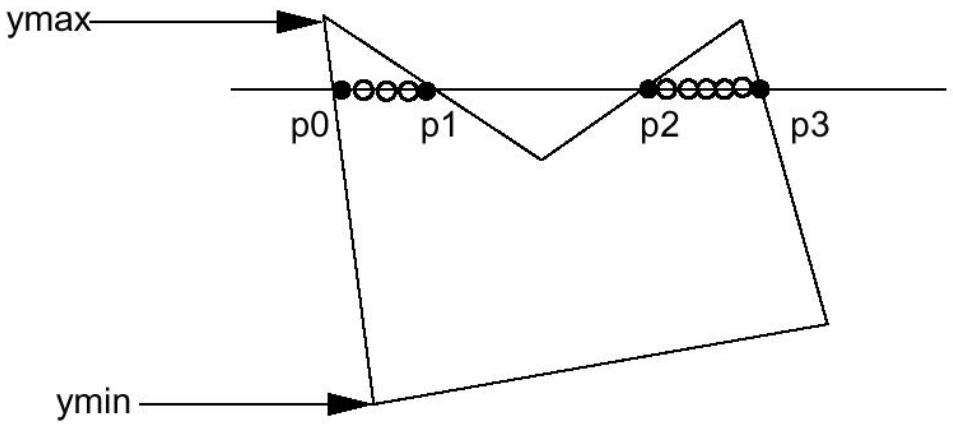
Scanline Fill Algorithm

- Intersect scanline with polygon edges
- Fill between pairs of intersections
- Basic algorithm:
  For $y = y_{min}$ to y_{max}
  1) intersect scanline y with each edge
  2) sort intersections by increasing x [p0,p1,p2,p3]
  3) fill pairwise (p0->p1, p2->p3, …)



Filled-Area Primitives

Basic Scan-Fill Algorithm

Some special need handling on polygons intersection.

- Where vertex is the meeting point of two edges.
- If the two edges of vertex is on the same side – filling is no problem.
- If the two edges are on the opposite site – will cause some filling problem

Special Handling

- Make sure we only fill the interior pixels

  Define interior:

  For a given pair of intersection points
  (Xi, Y), (Xj, Y)

  -> Fill ceilling(Xi) to floor(Xj)

  important when we have polygons adjacent
  to each other.

Special Handling (cont.)

- Intersection has an integer X coordinate

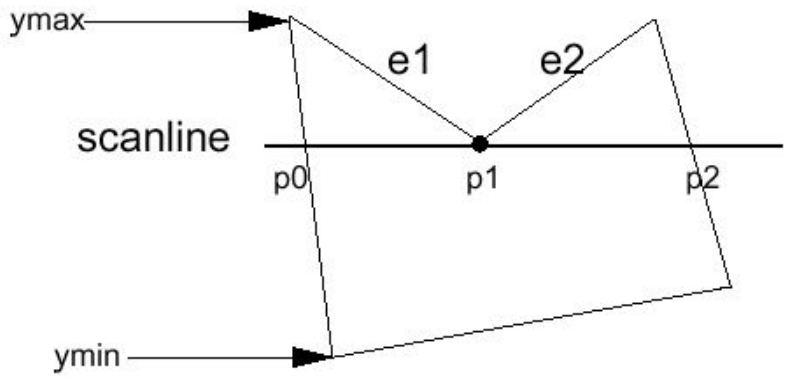
  ->if Xi is integer, we define it to be interior

  ->if Xj is integer, we define it to be exterior
  (so don’t fill)

Special Handling (cont.)

- Intersection is an edge end point

Case 1



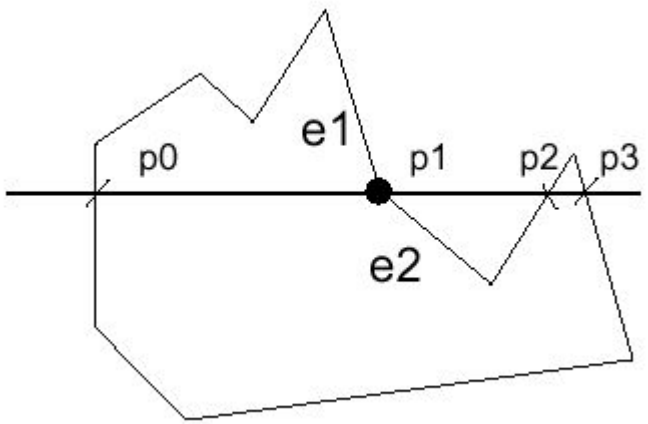
Intersection points: (p0, p1, p2) ?

->(p0,p1,p1,p2) so we can still fill pairwise

->In fact, if we compute the intersection of the scanline with edge e1 and e2 separately, we will get the intersection point p1 twice. Keep both of the p1.

Special Handling (cont.)

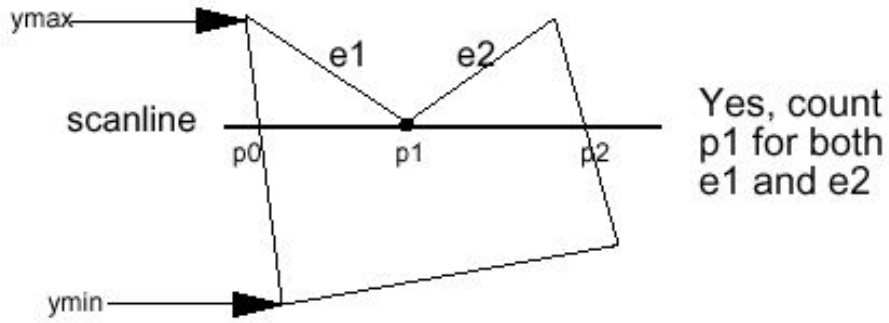
Case 2

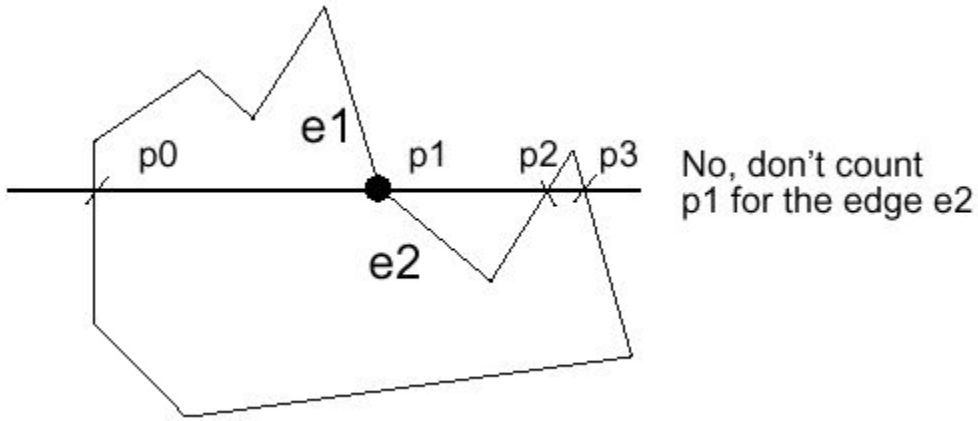


However, in this case we don’t want to count p1 twice (p0,p1,p1,p2,p3), otherwise we will fill pixels between p1 and p2, which is wrong.

Special Handling (cont.)

Summary: If the intersection is the y_{min} of the edge's endpoint, count it. Otherwise, don't.





Basic Structure

- For $y=y_{min}$ to y_{max}
  - Intersect scanline with each edge
  - Sort Intersections by increasing x
  - Fill Pairwise

How to make sure it works correctly and fast?

Two important features:

- Scanline Coherence: Values don't change much from one scanline to next– the coverage (visibility) of a face on one scanline typically differs little from the previous one.
- Edge Coherence: Edges intersected by scanline 's' are typically intersected by scanline 's+1'.

Implementation using integer arithmetic

(x_{k+1}, y_{k+1}) $y_{k+1} = y_k + 1$

$m = (y_{k+1} - y_k)/(x_{k+1} - x_k)$

(x_k, y_k) Intersection for next scanline is $x_{k+1} = x_k + 1/m$ obtained as x_{k+1} = round(x_k + 1/m)

Eg. m=Δy/ Δx=7/3

Set counter, c =0; & counter-increment, Δc= Δx=3

For the next three scanlines, successive values of c are: 3, 6, 9…

At 3rd scanline, c> Δy

Then x is incremented by 1 only at 3rd scanline and set as c=c- Δy= 9-7=2

Repeat above steps till y_k reaches y_{max}.

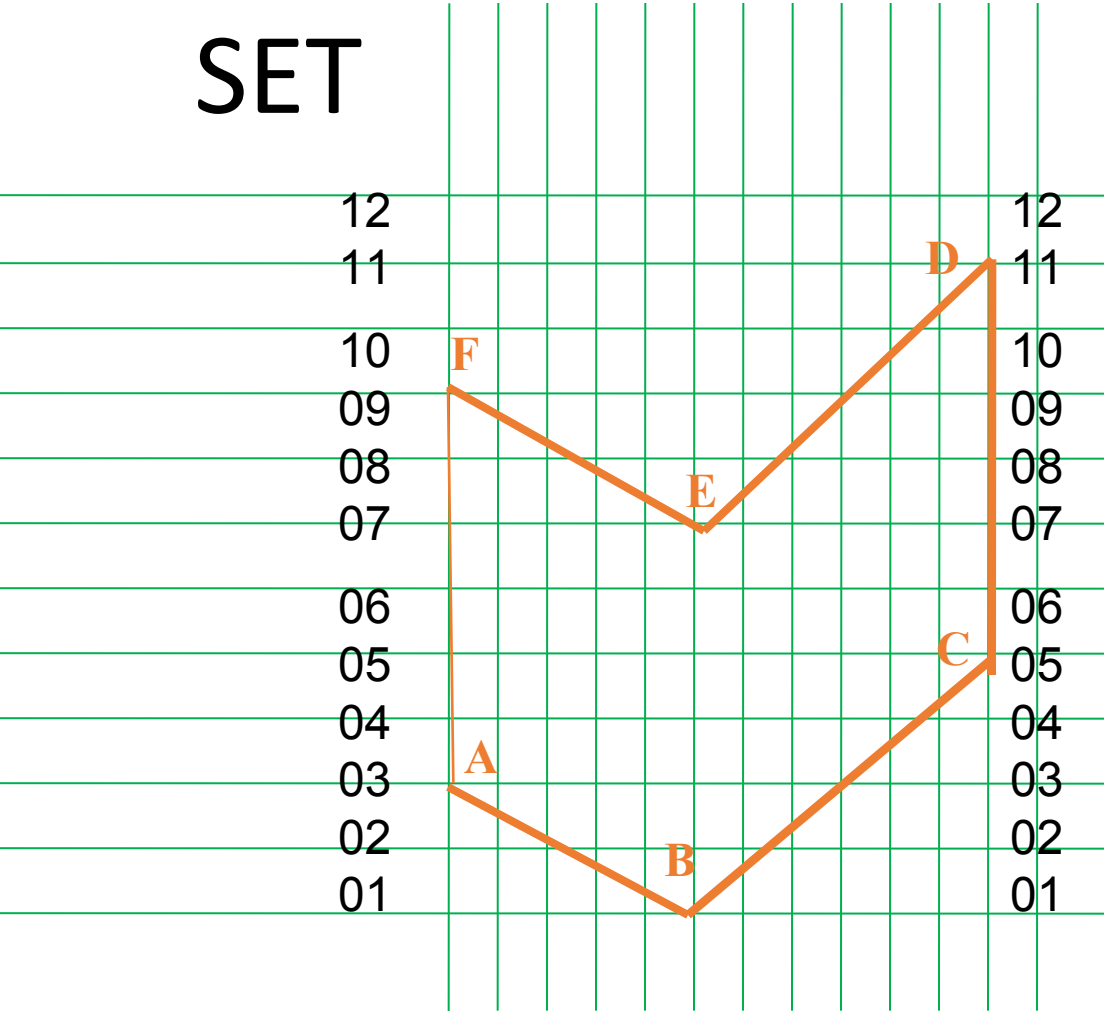
Data Structure used

- SET (Sorted Edge Table)
  - Built using bucket sort, with as many buckets as there are scanlines.
  - All edges are sorted by their y_{min} coordinate, with a separate y bucket for each scanline.
  - Within each buicket, edges are sorted by increasing x of y_{min} point.
  - Only non-horizontal edges are stored. Store these edges at the scanline position in the SET.
  - Edge Structure: Sample record for each scanline: (ymax, xmin, $\Delta x/\Delta y$, pointer to next edge)

Data Structure used

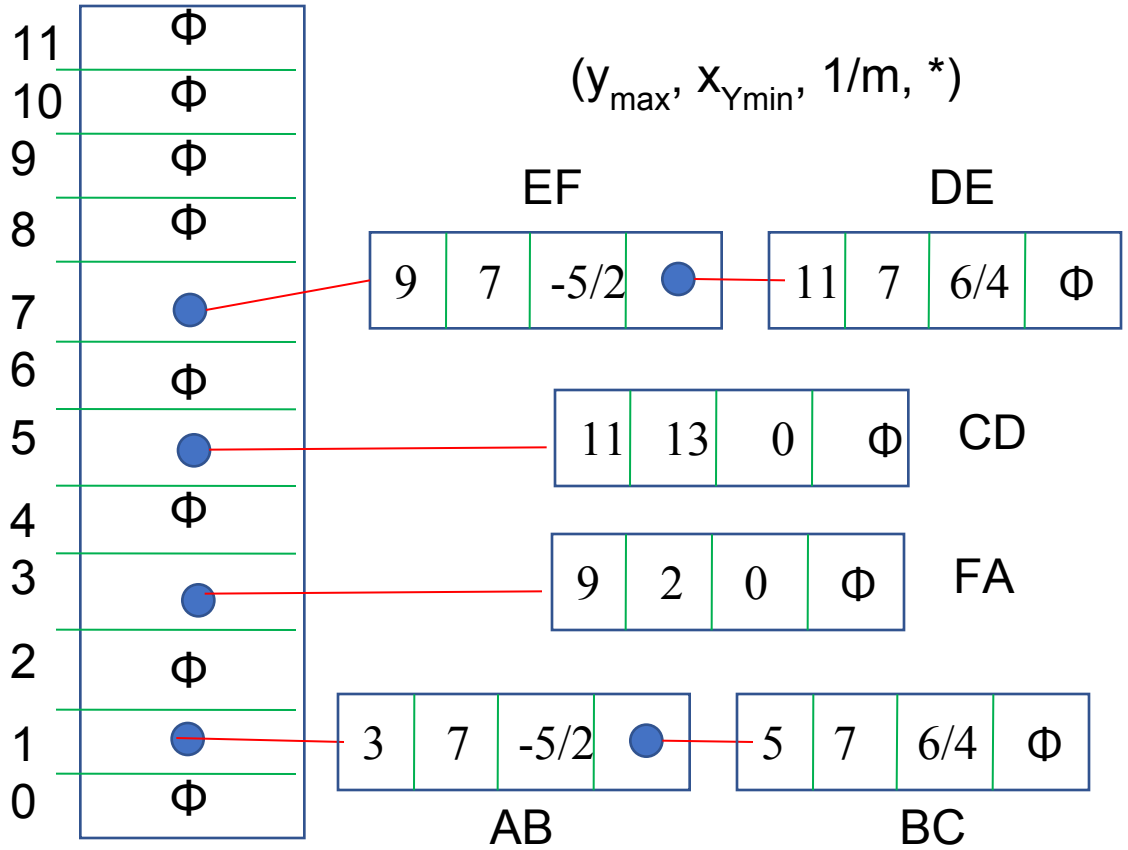
- AEL (Active Edge List)
  - Contains all edges crossed by a scanline at the current stage of iteration.
  - Contains list of edges that are active for this scanline, sorted by increasing x intersections.
  - Also called active edge table (AET).
  - AEL structure: (y_{max}, x, $\Delta x/ \Delta y$, pointer to next edge)

SET



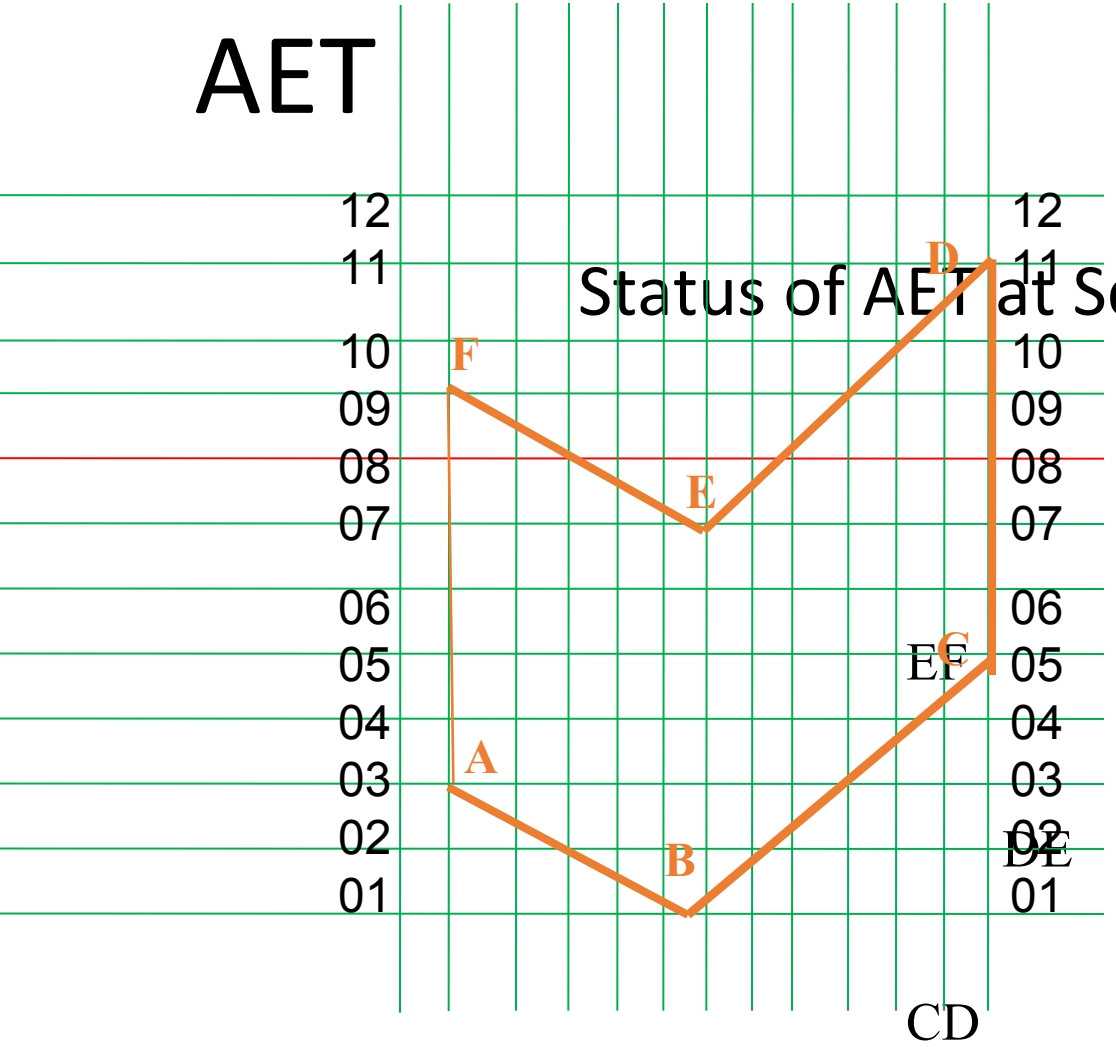
Anti-clockwise traversal

BUCKET-sorted Edge Table for Polygon

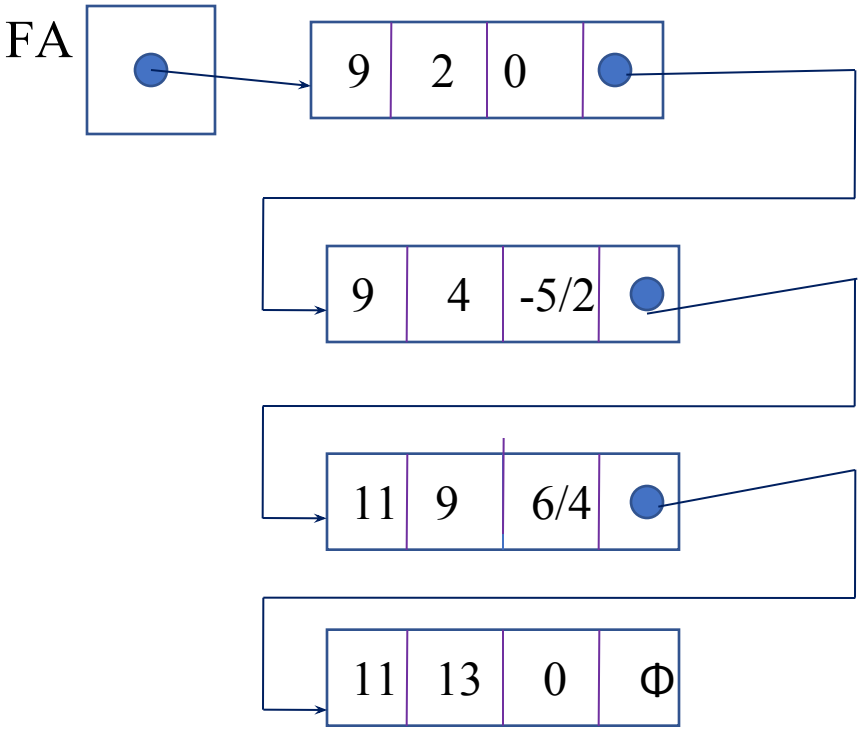


AET

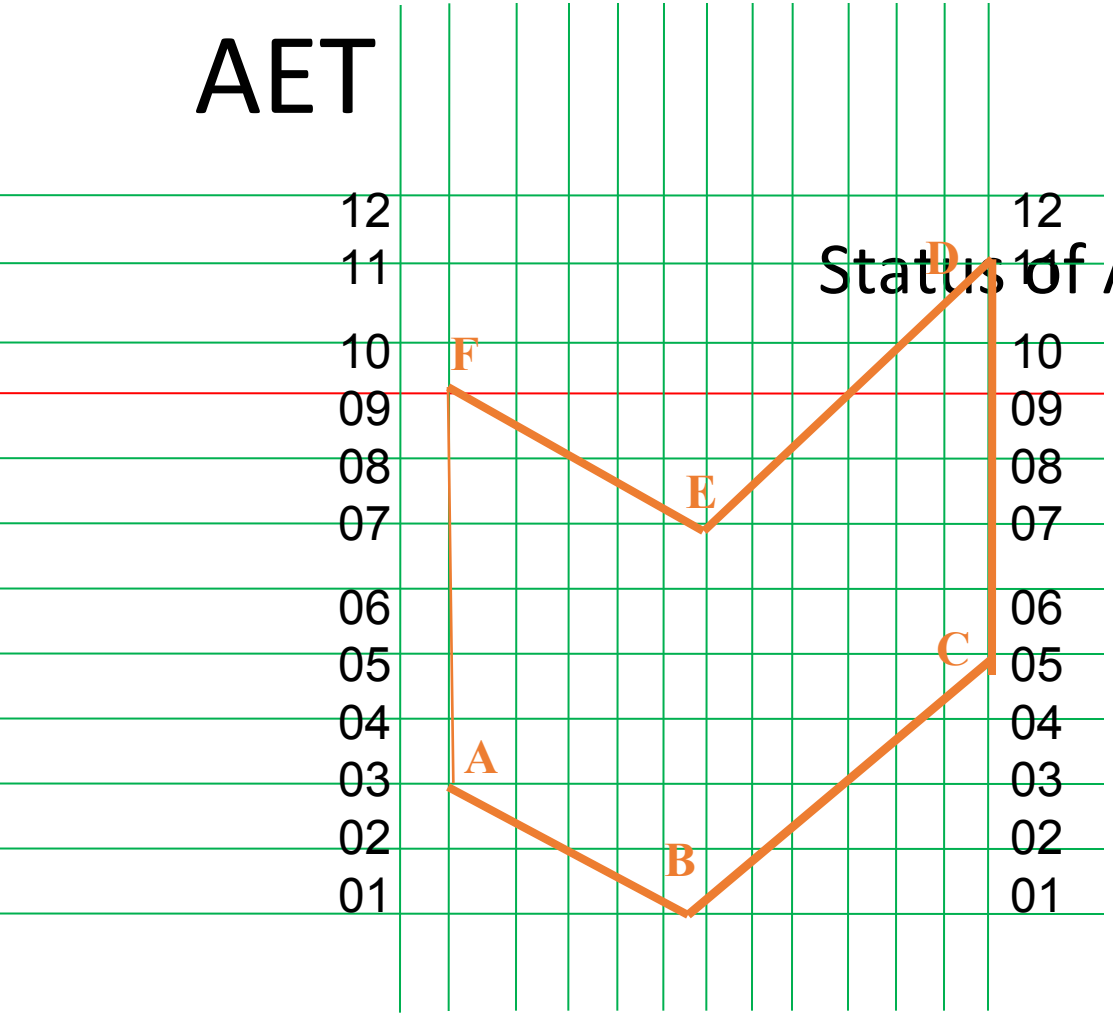
Status of AET at Scanline 8



$(y_{max}, x, 1/m, *)$



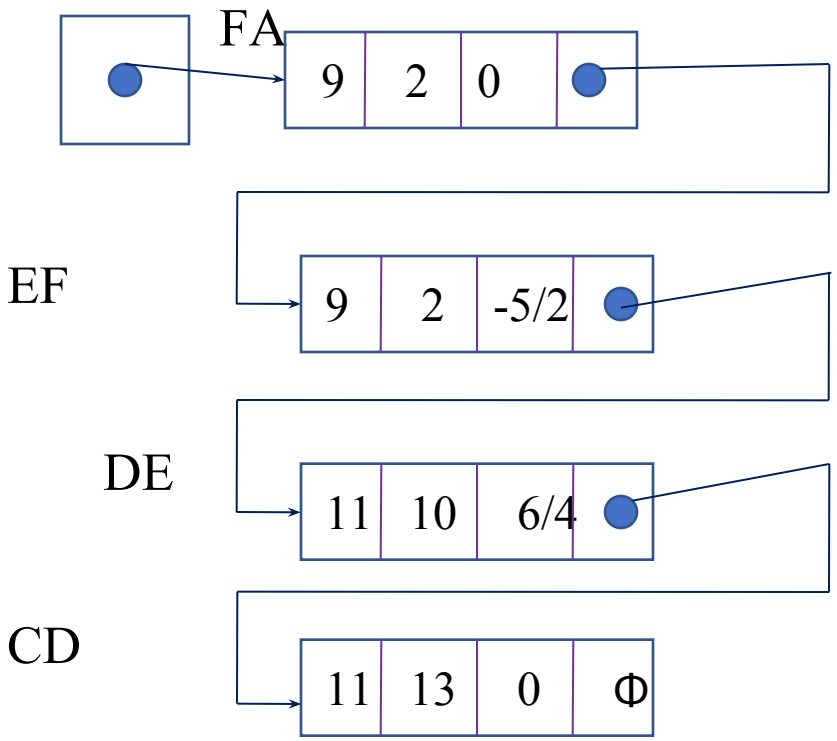
AET



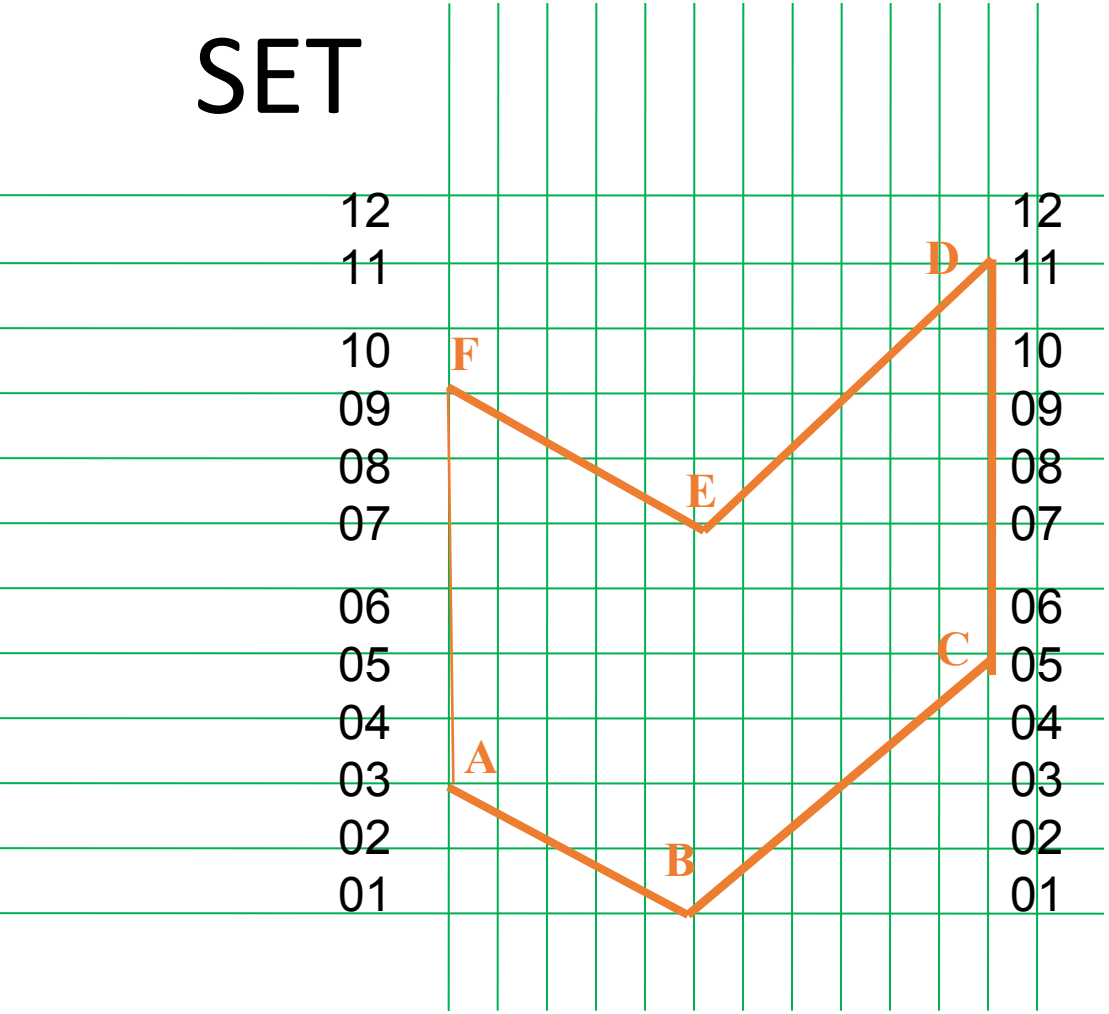
$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF

Status of AET at Scanline 9

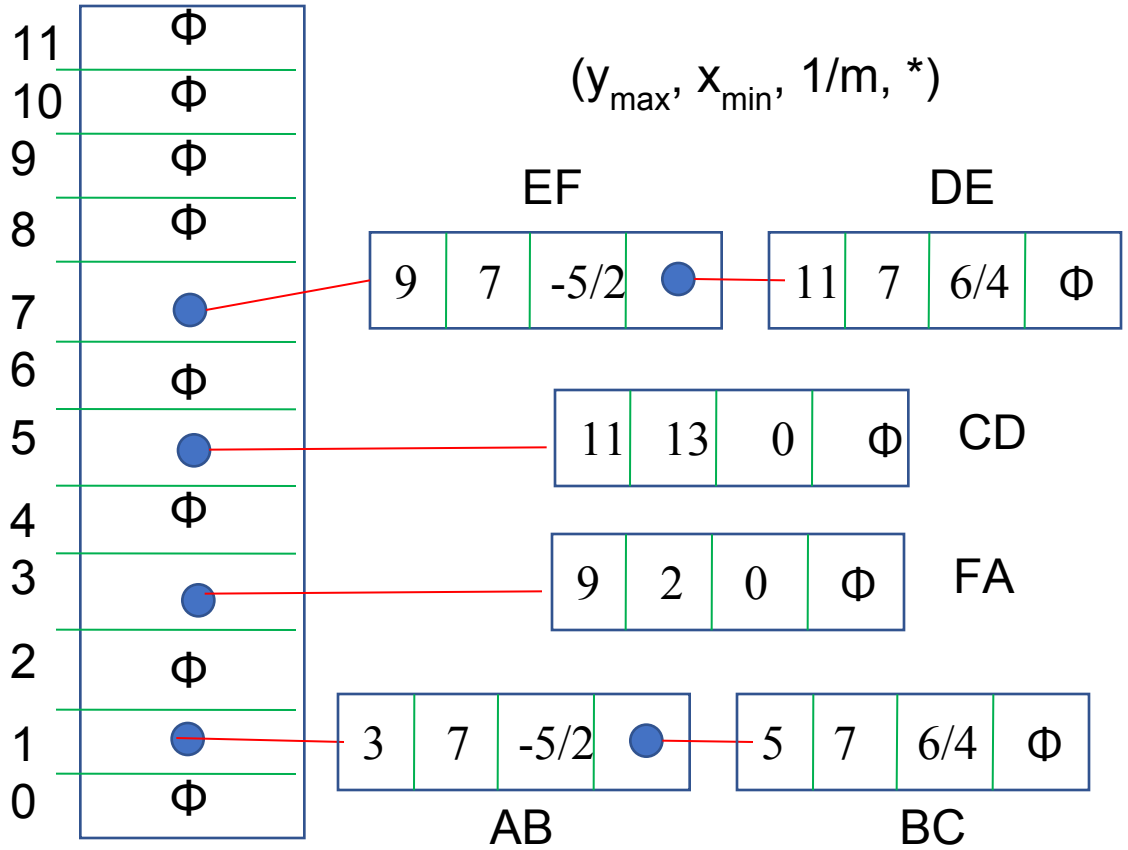


SET

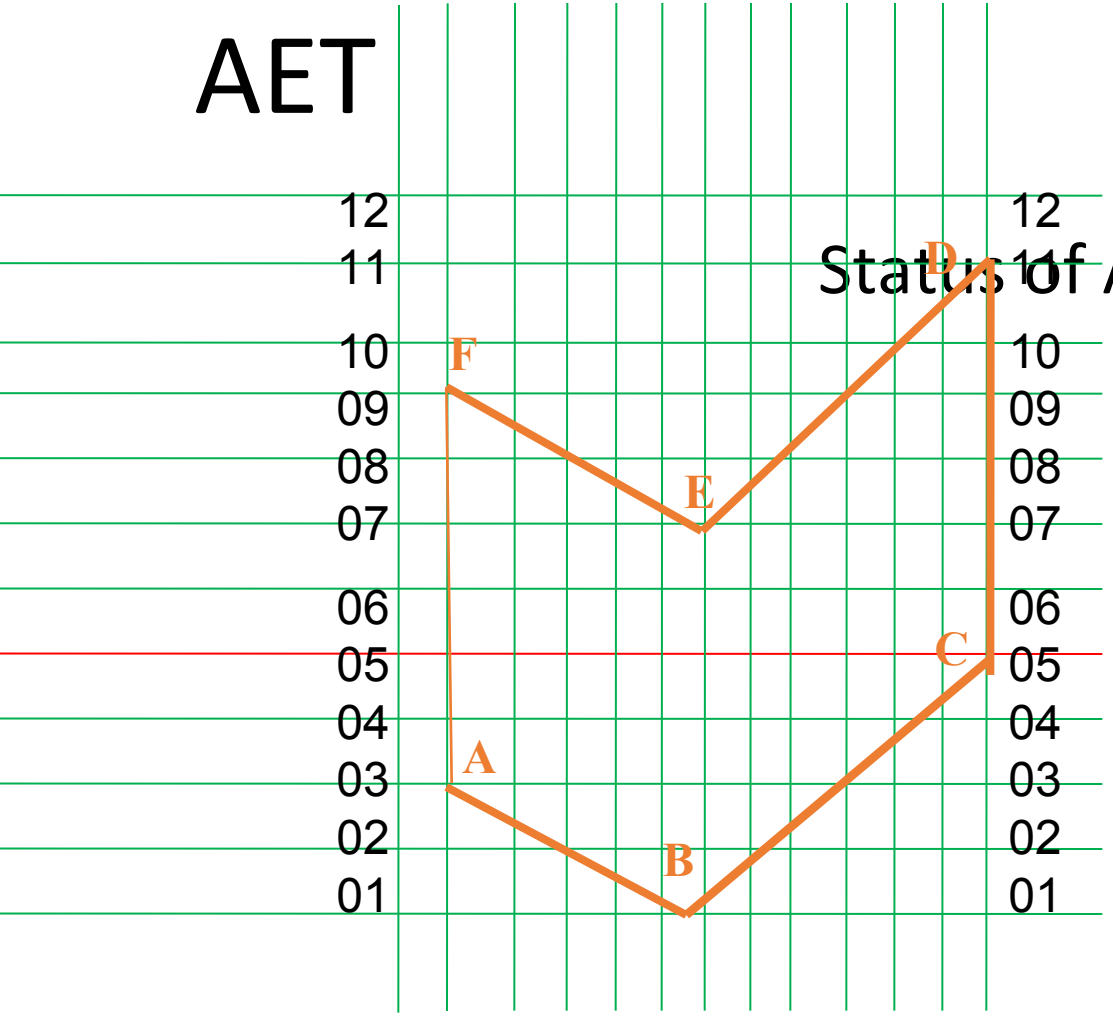


Anti-clockwise traversal

BUCKET-sorted Edge Table for Polygon



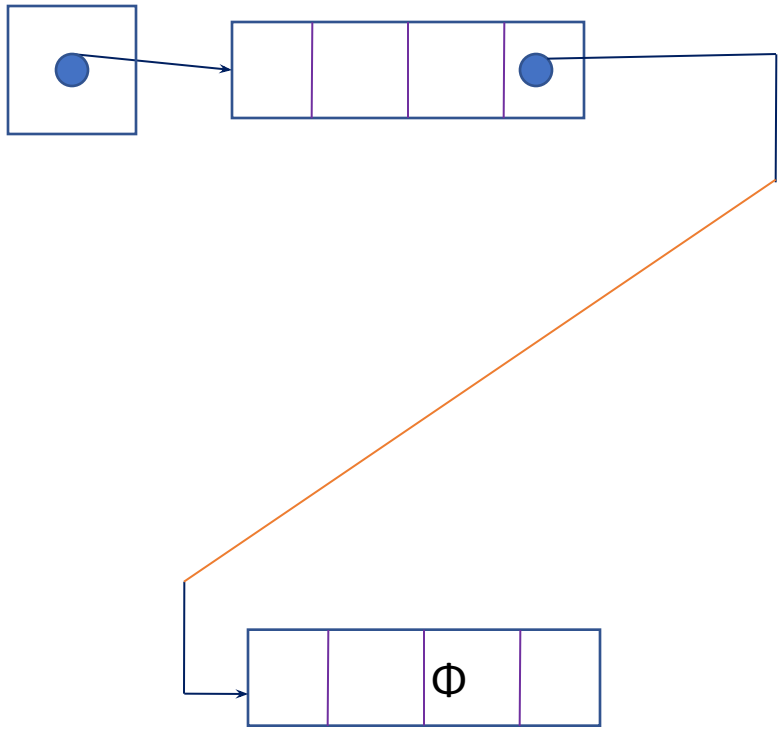
AET



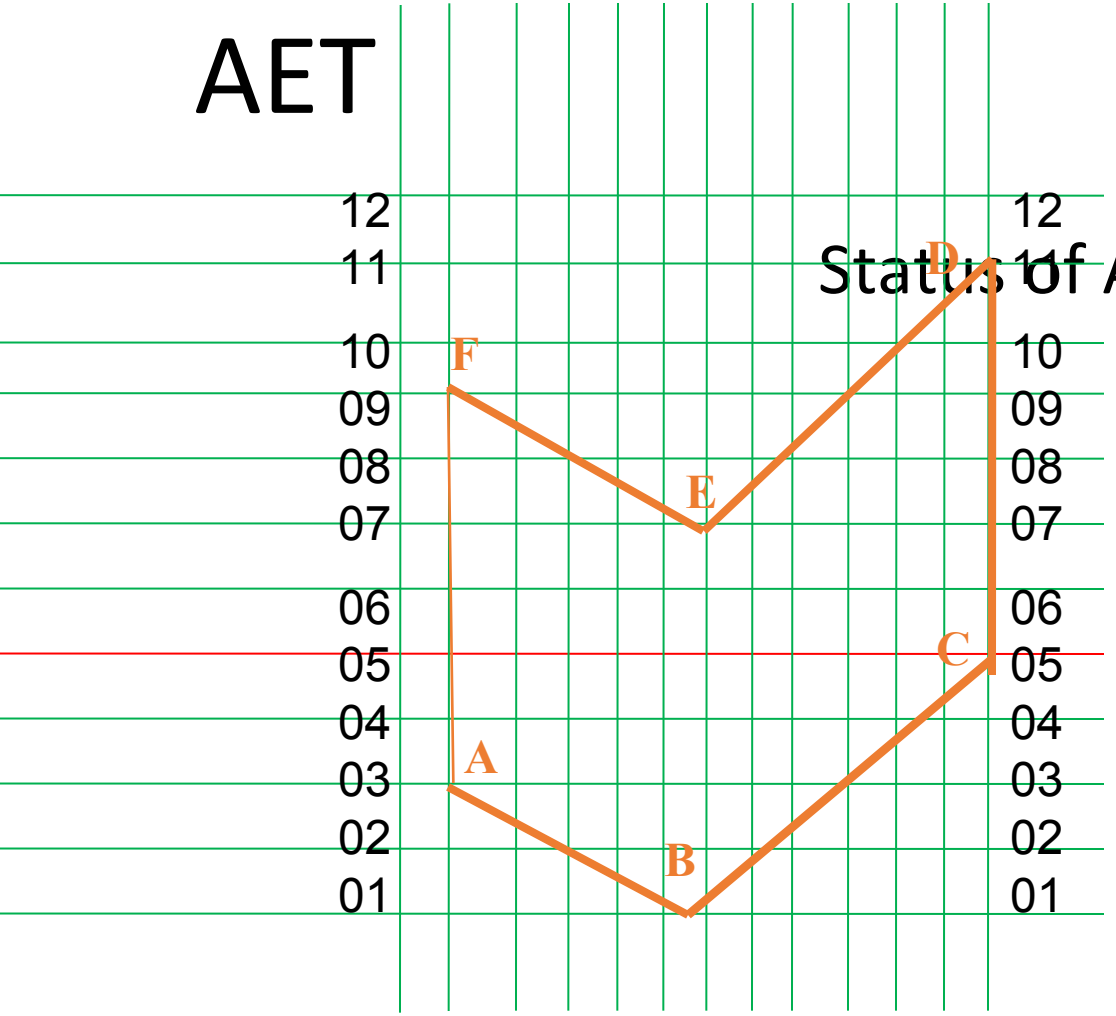
Status of AET at

$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF



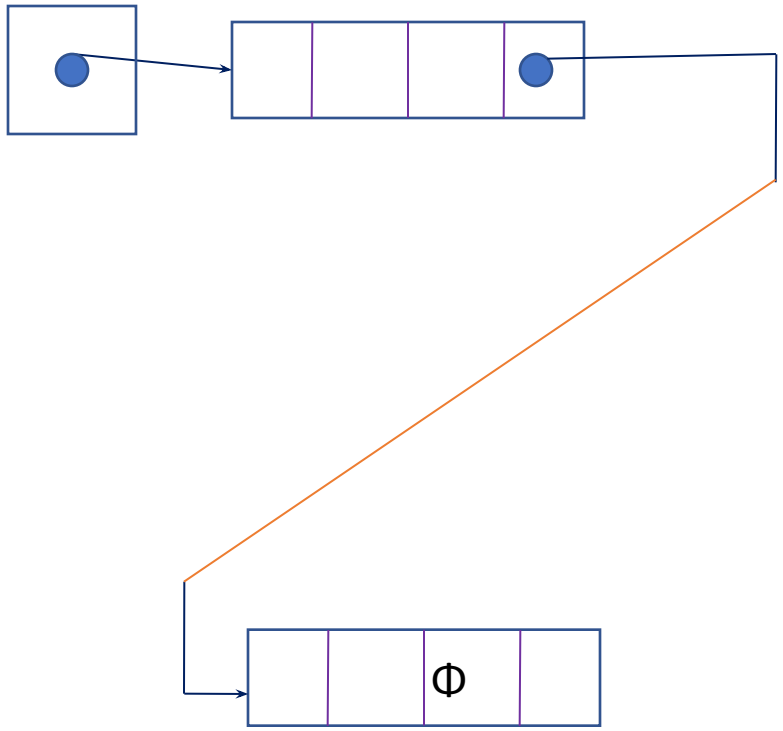
AET



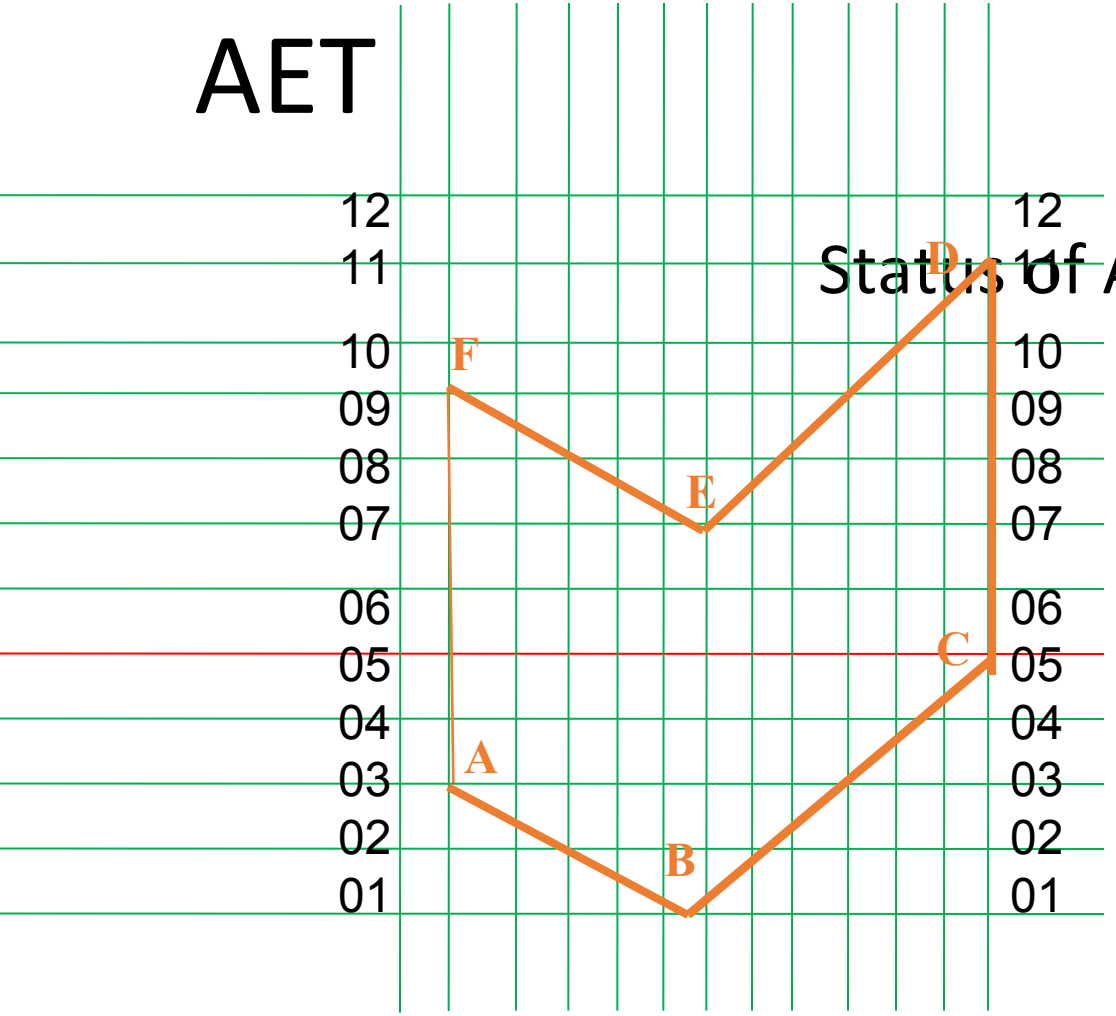
Status of AET at

$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF



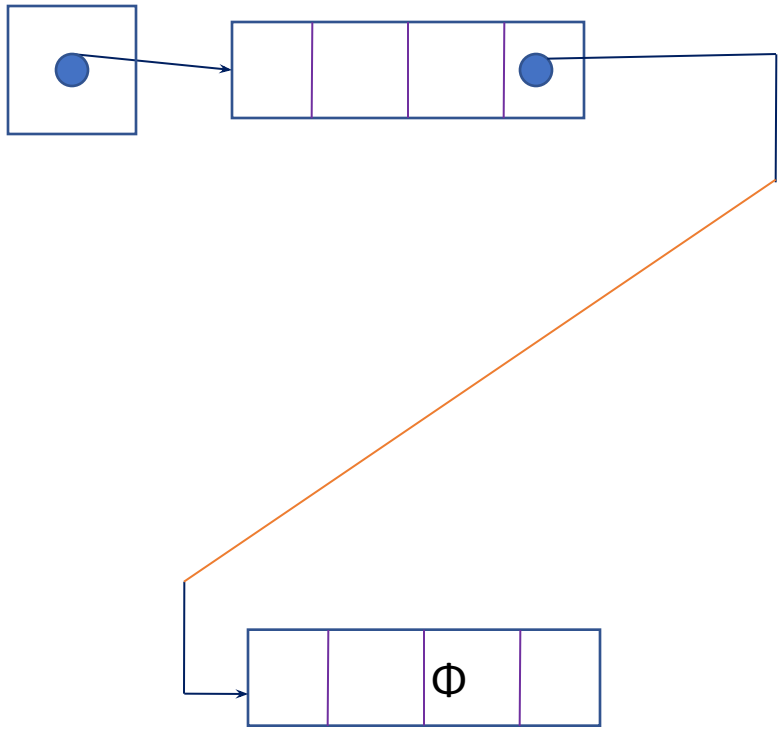
AET



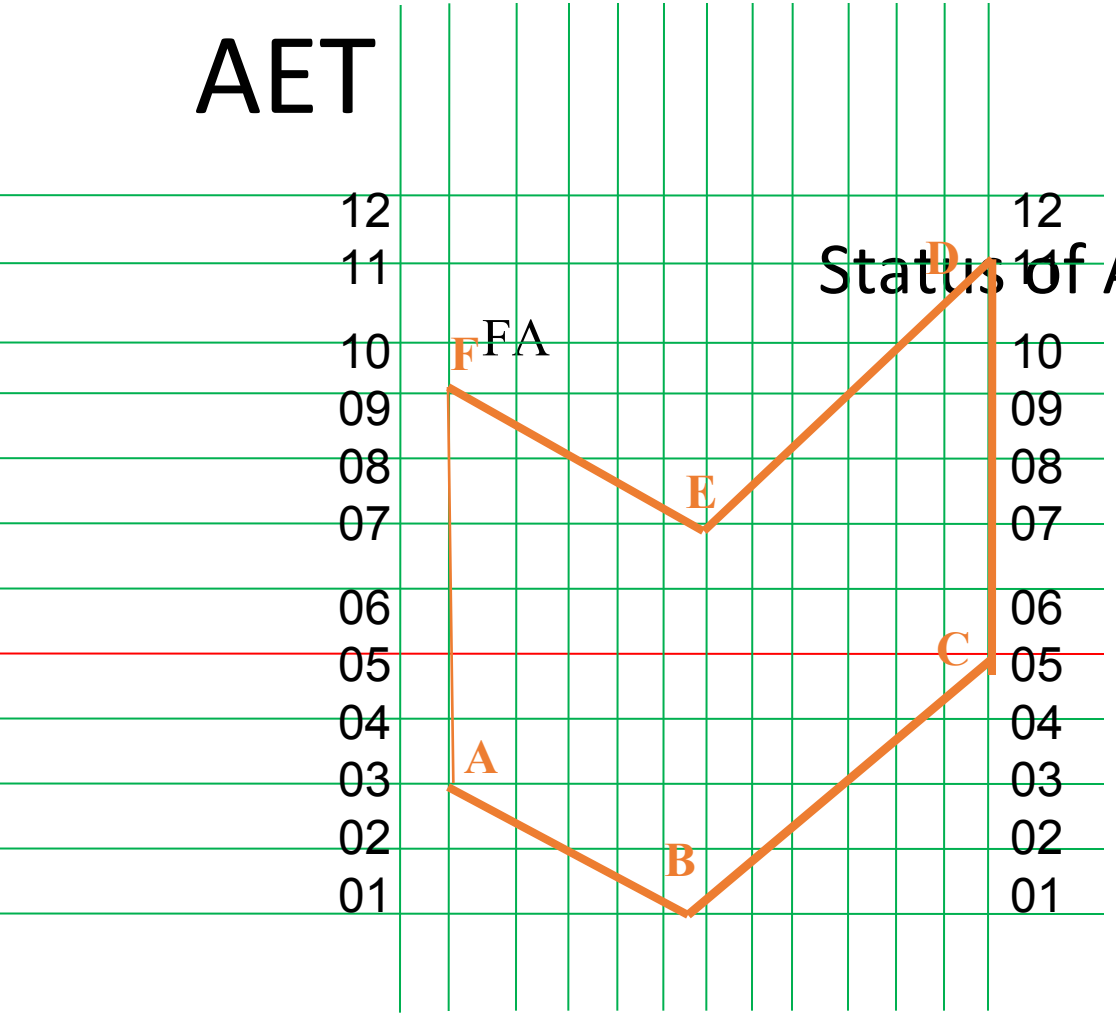
Status of AET at

$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF



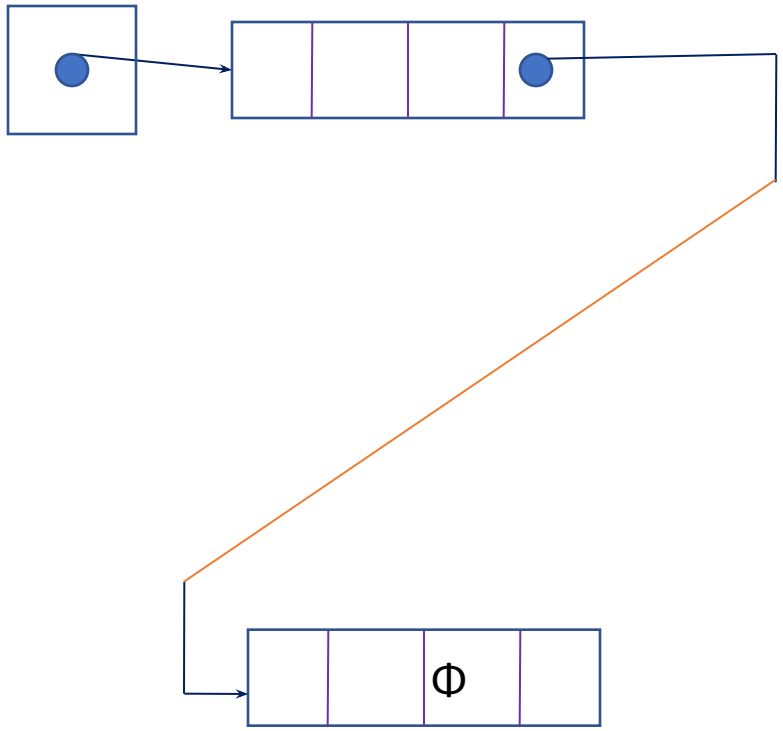
AET



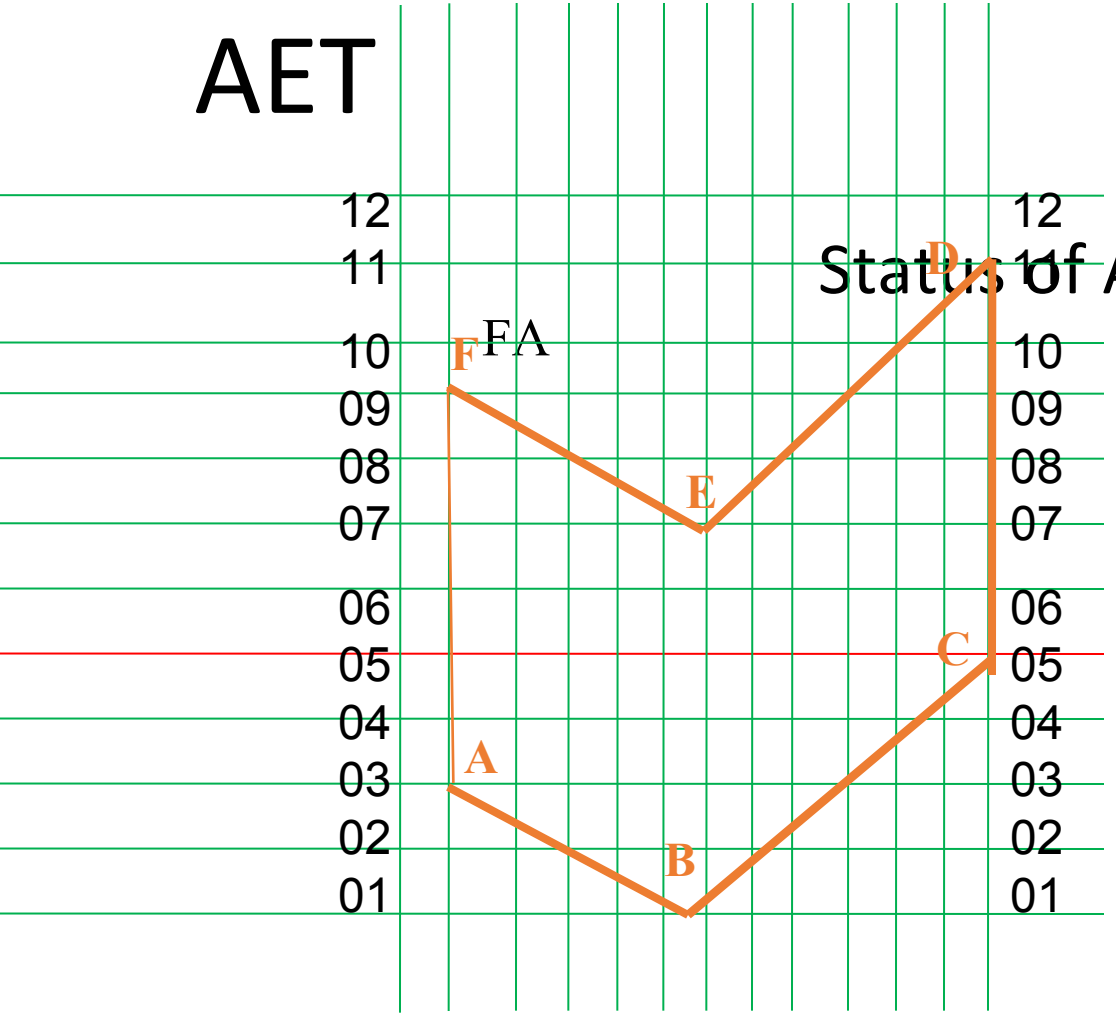
Status of AET at

$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF



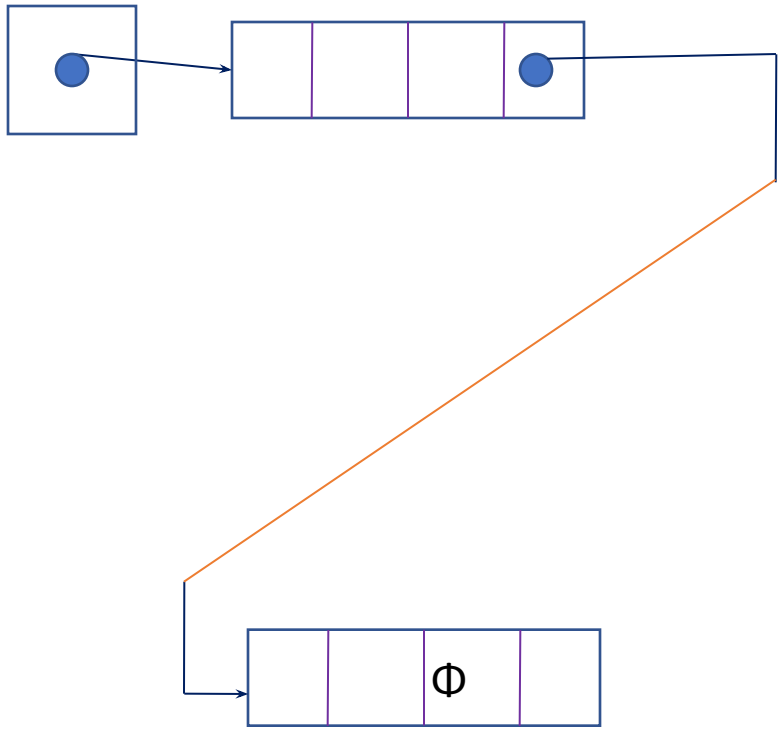
AET



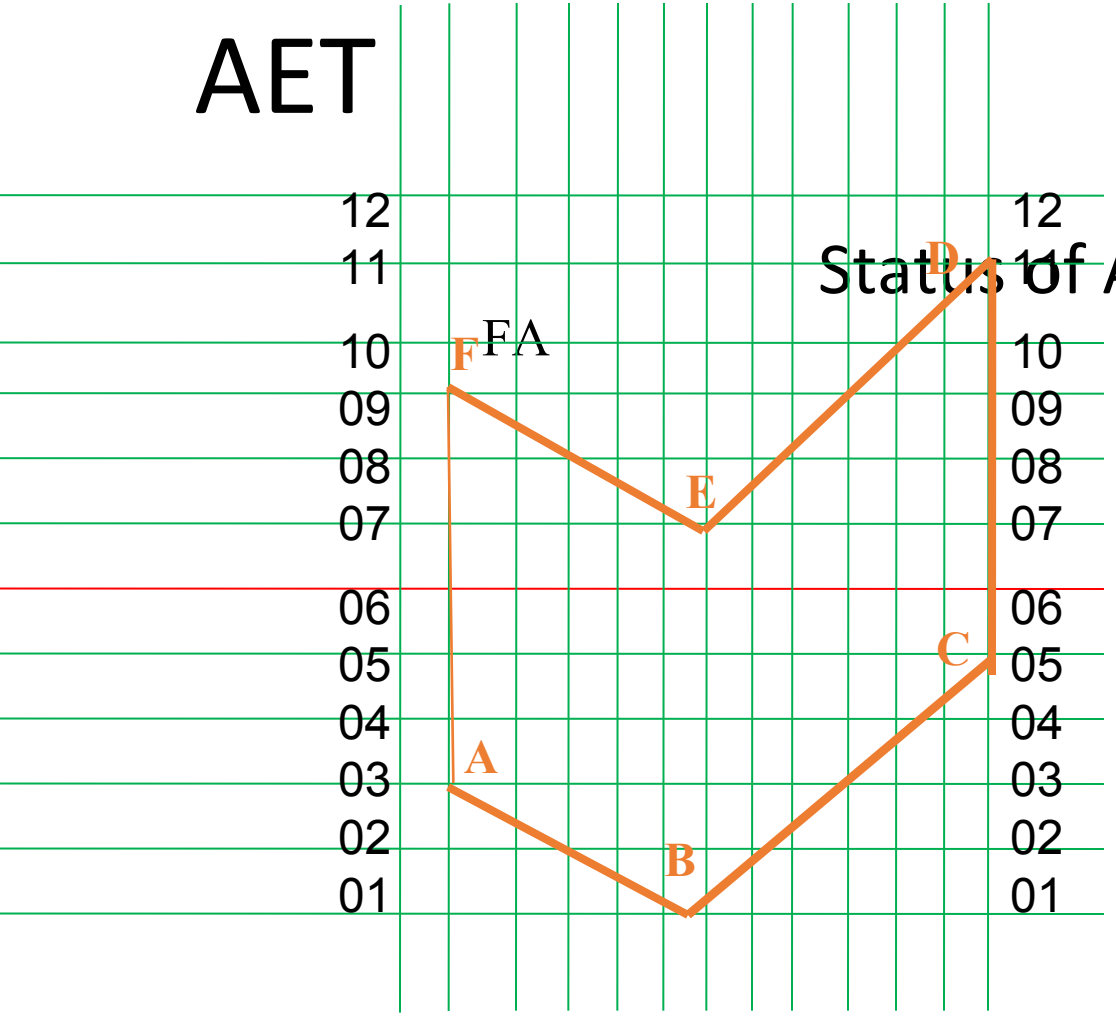
Status of AET at

$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF



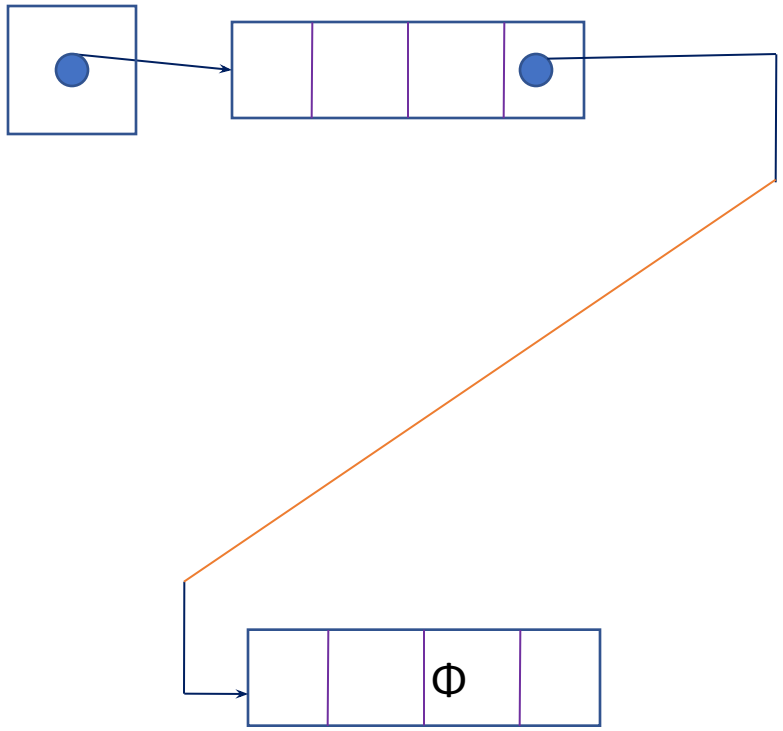
AET



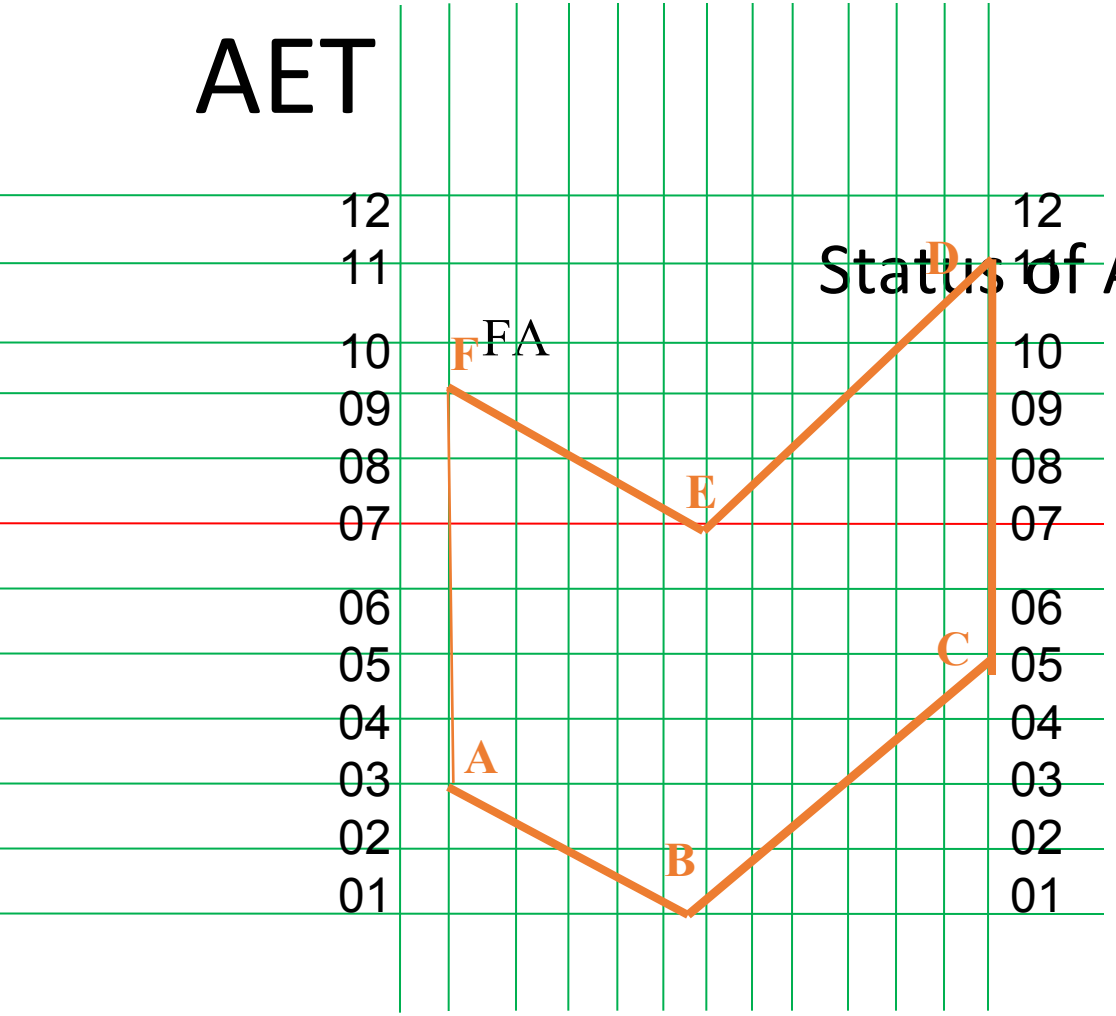
Status of AET at

$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF



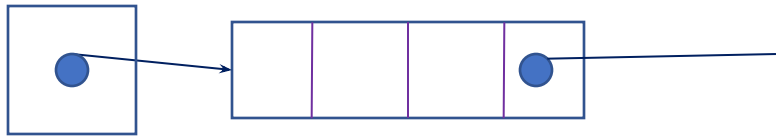
AET

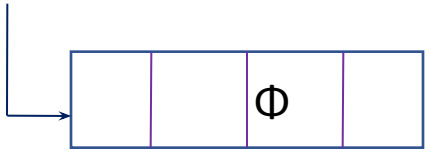


Status of AET at

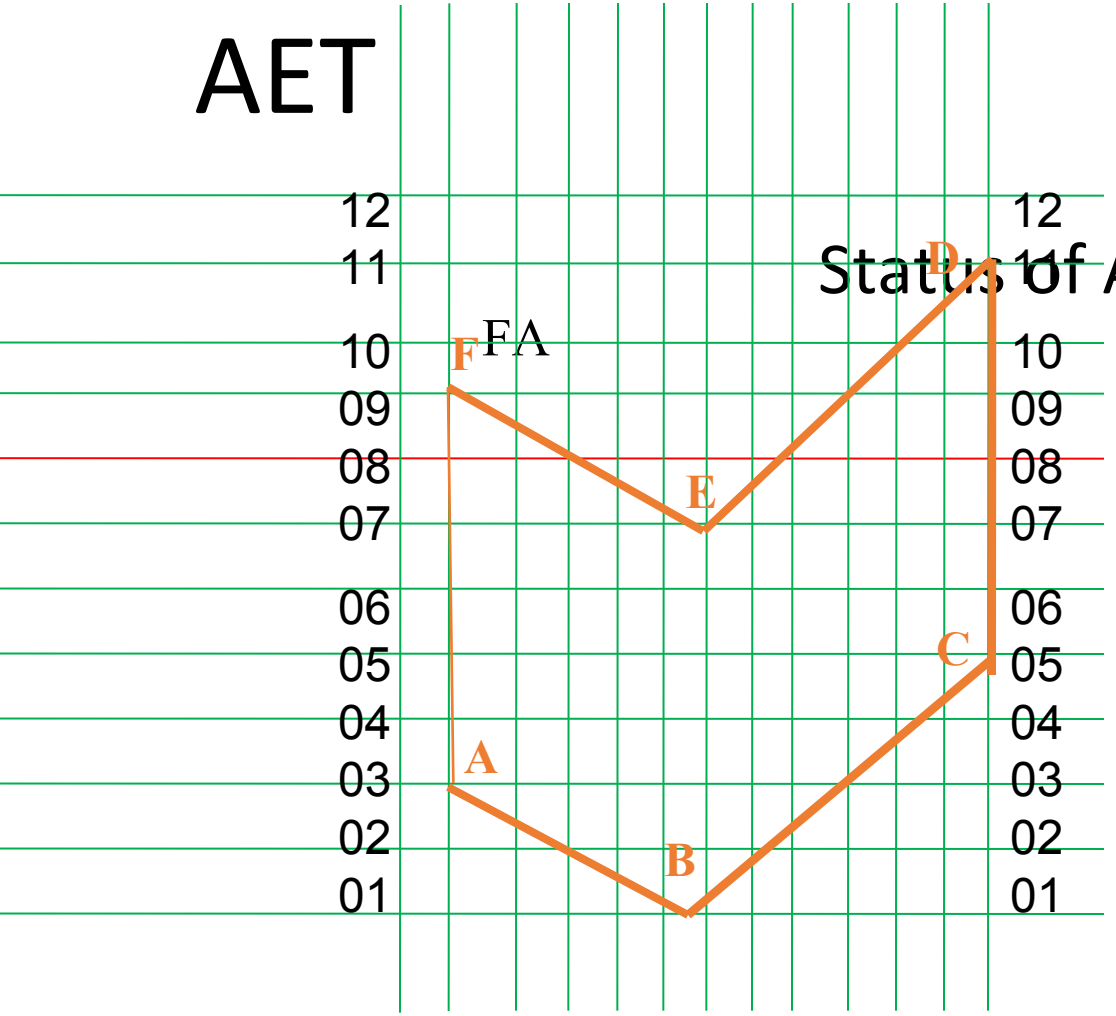
$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF





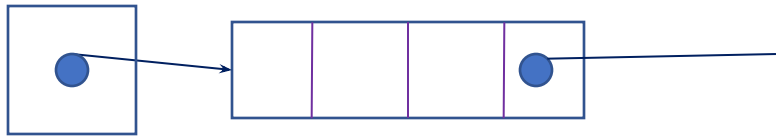
AET

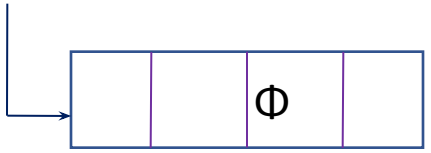


Status of AET at

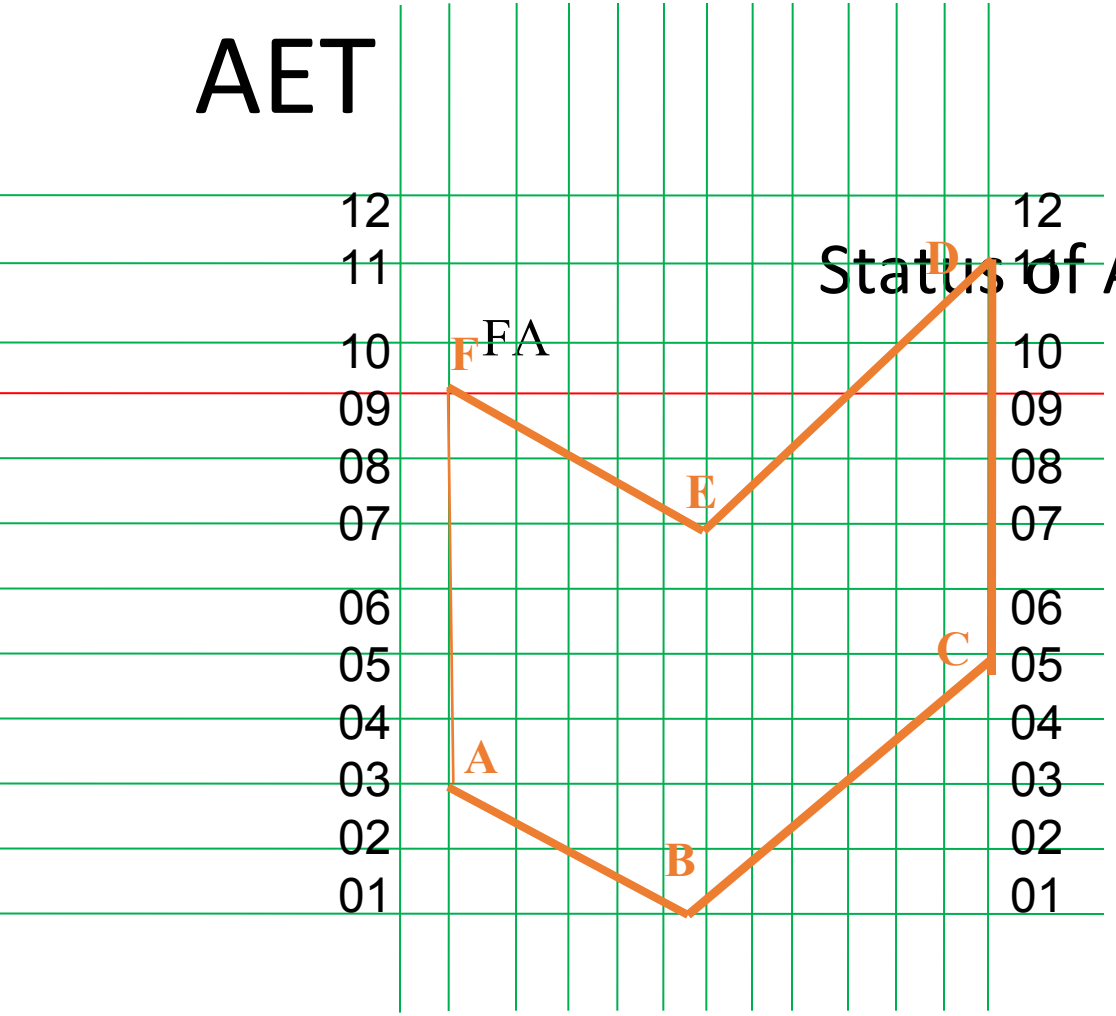
$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF





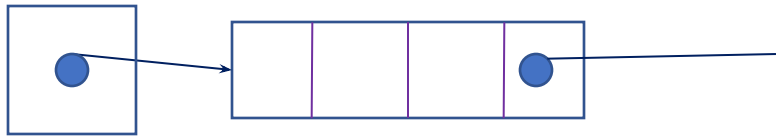
AET

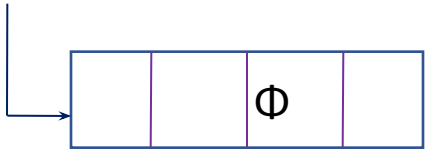


Status of AET at

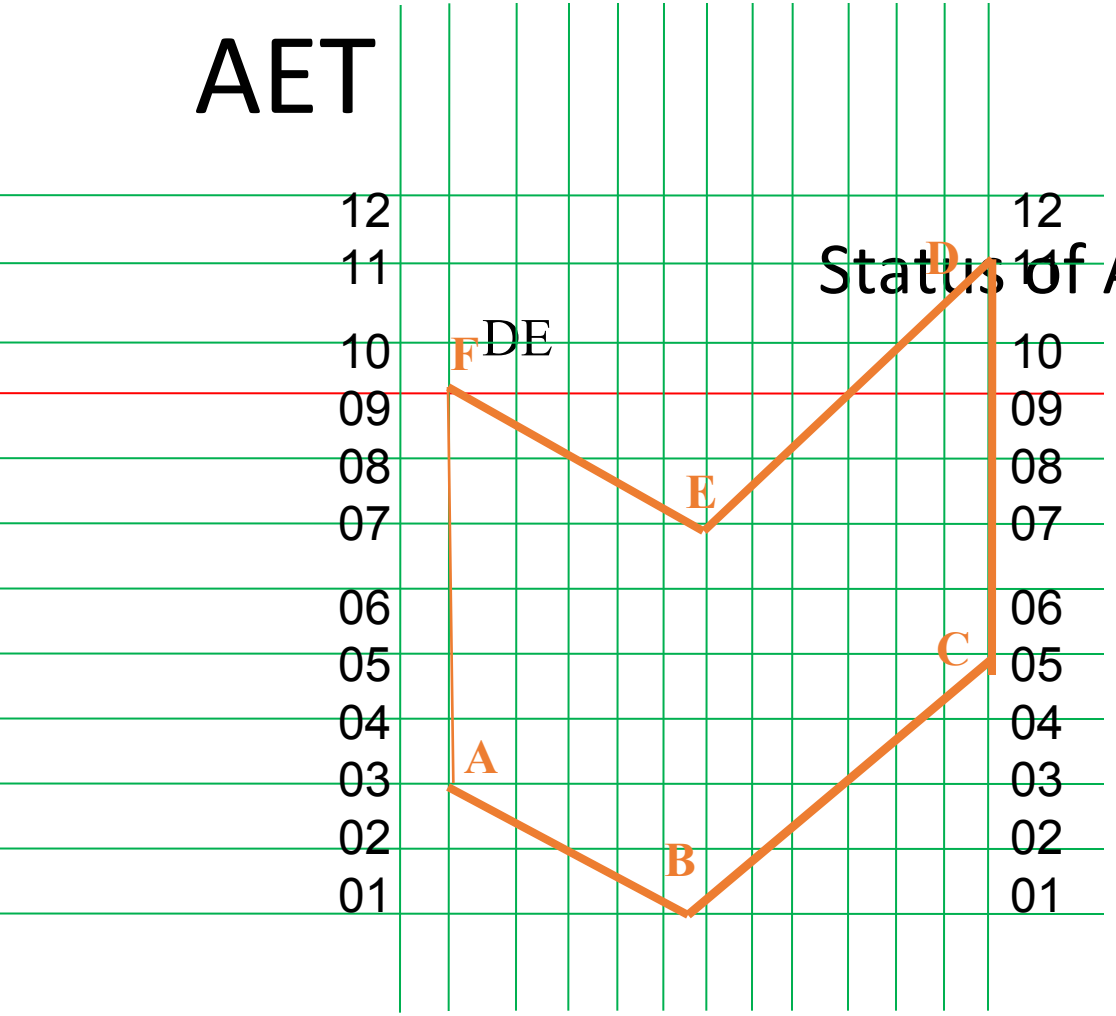
$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF





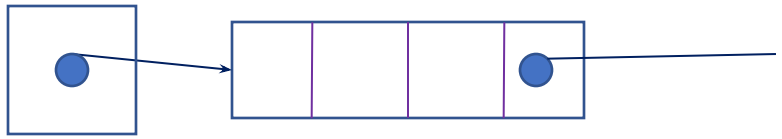
AET

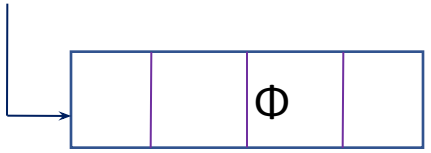


Status of AET at

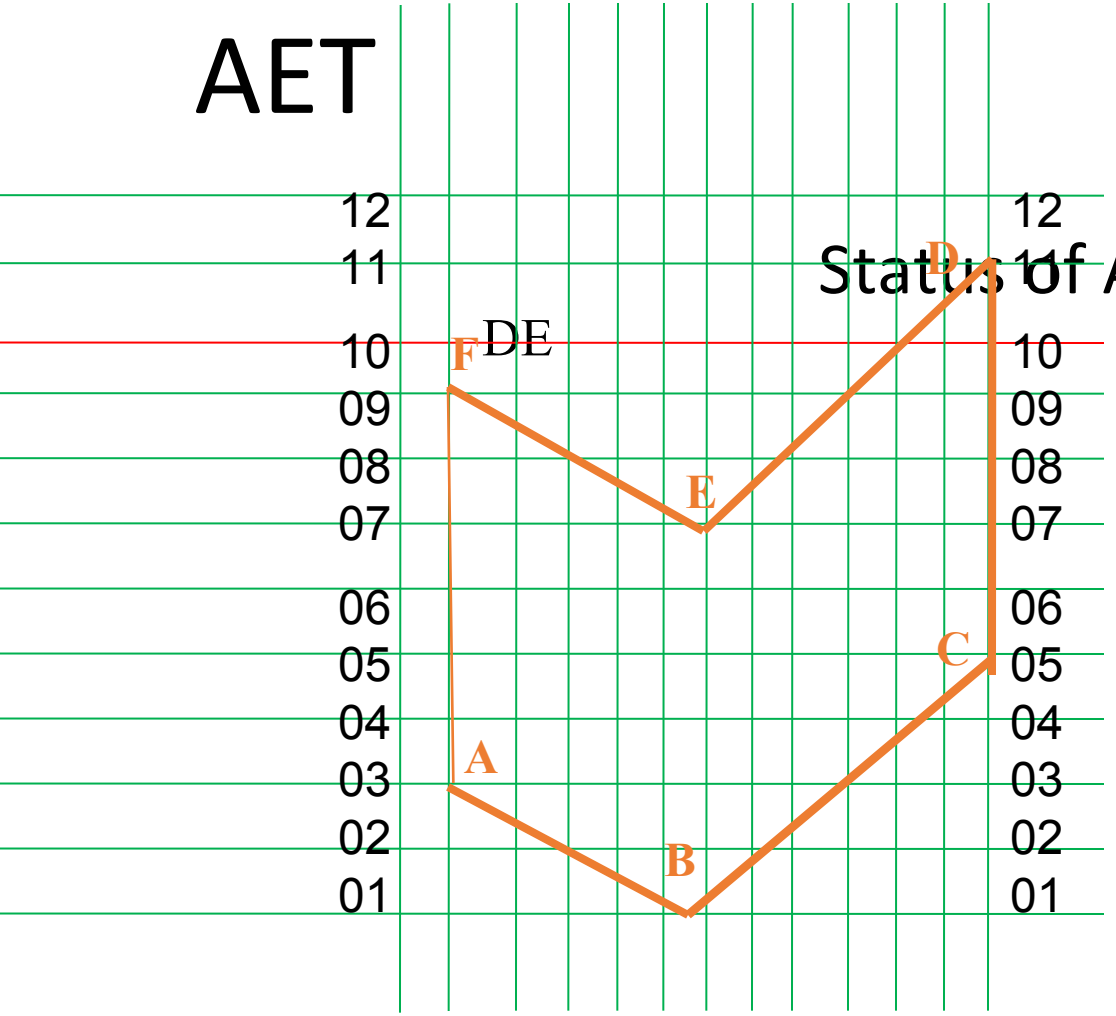
$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF





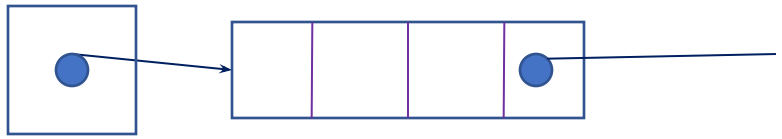
AET

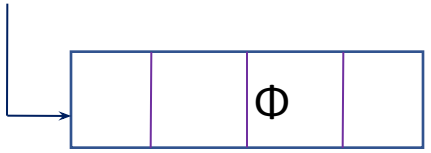


Status of AET at

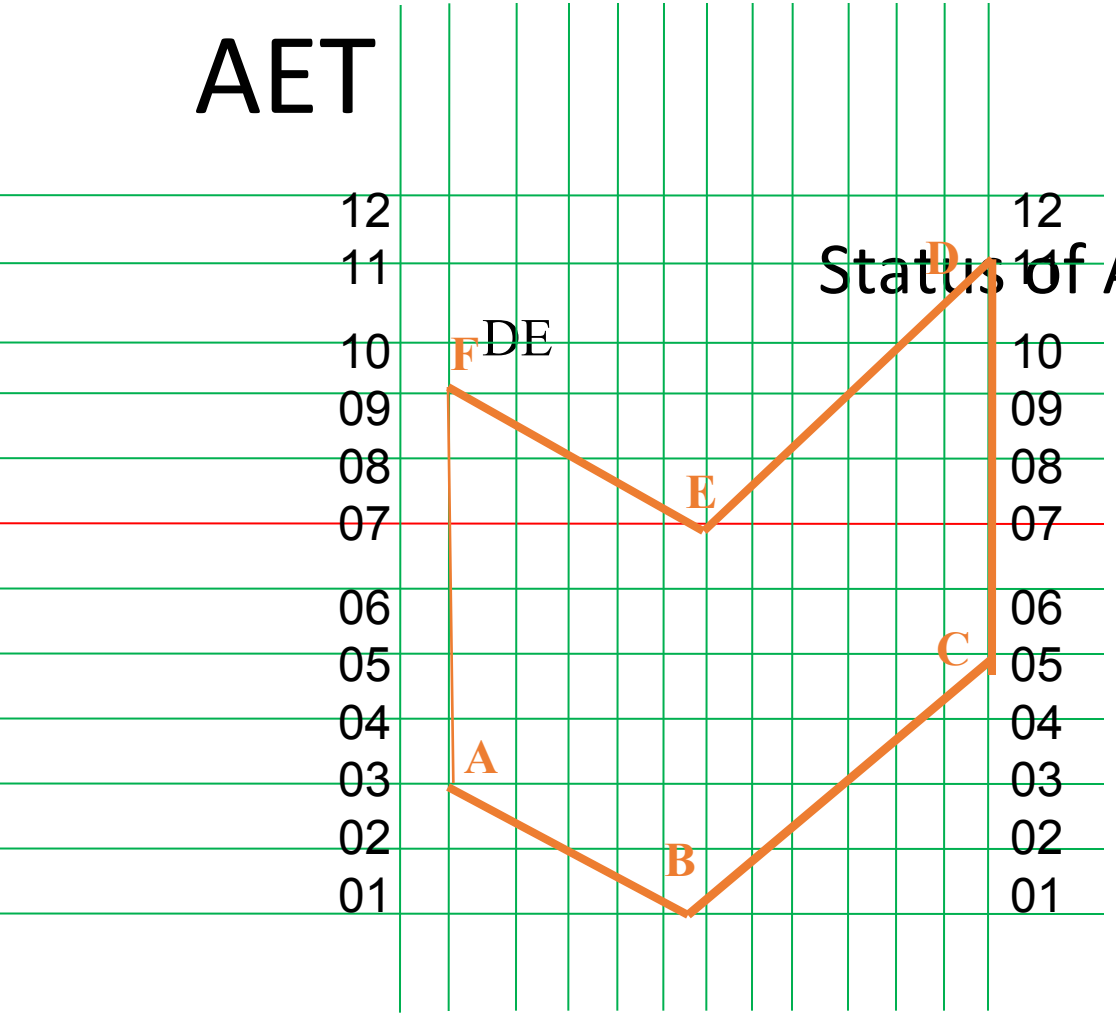
$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF





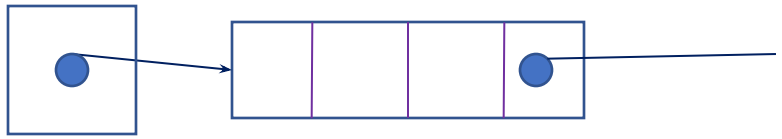
AET

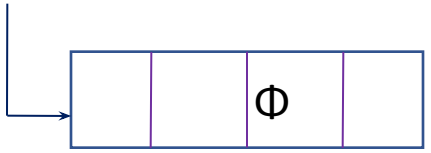


Status of AET at

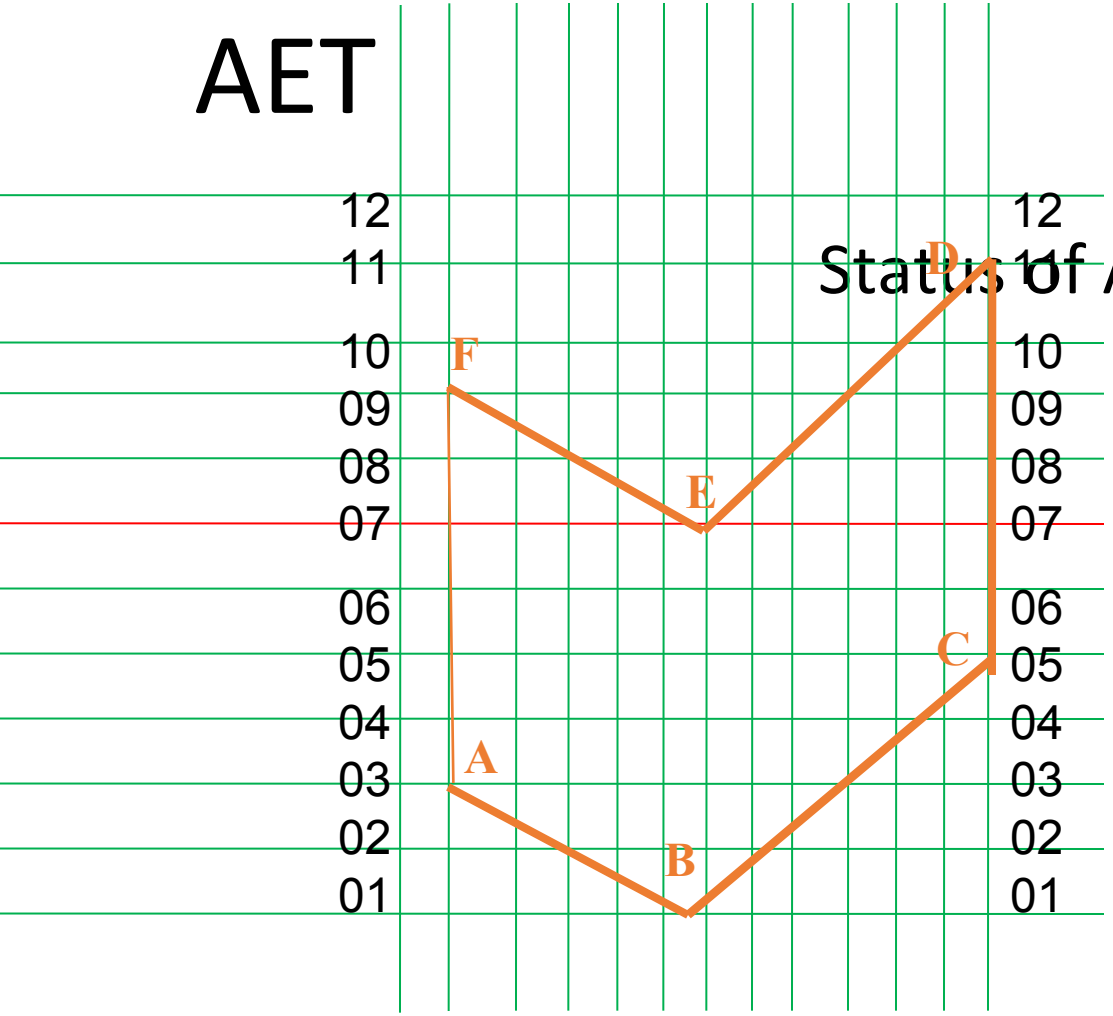
$(y_{max}, x, 1/m, *)$

Now, y=ymax, throw out FA & EF





AET



Status of AET at

(y_{max}, x, 1/m, *)
Now, y=ymax, throw out FA & EF

PRECAUTIONS

- Intersection has an integer y coordinate
- If this point is the y_{min} of the edges endpoint count it.
- Count only non-horizontal edges
- During each iteration process with a scanline, the AET is updated
- For each scanline, AET keeps track of the set of edges it has to intersect & stores the intersection points. The sorting of the entries is w.r.t. the x-intersection values.

Processing steps

1. Set y to smallest y in SET entry (first non-empty bucket)
2. Initialize AET to be empty
3. Repeat until both AET & SET are empty
   I. Move from SET bucket y to AET, those edges whose $y_{min} = y$
   II. Sort AET on x (Simple as SET is pre-sorted)
   III. Fill pixels on scanline y using pairs of x-coordinates from AET
   IV. Increment scanline by 1
   V. Remove from AET those entries for which $y=y_{max}$ (edges not involved)
   VI. For each non-vertical edge in AET, update x for new y
4. End loop

Algorithm

1. Construct Sorted Edge Table (SET)
2. Y_{min} =min (all y in the ET)
3. AET = NULL
4. For $y=y_{min}$ to y_{max}
    I. Merge-sort SET[y] into AET by x value
    II. Fill between pairs of x in AET
    III. For each edge in AET
        I. If $edge.y_{max} = y$
            I. Remove edge from AET
        II. Else
            I. edge.x = edge.x + dx/dy
        III. End if
    IV. Sort AET by x valye
    V. End

Attributes of Output Primitives

Definition

Parameter that affects the way a primitive will be displayed

Line Attribute

. Type

. Width

. Color

. Pen & Brush

Line Attribute

Type

. Solid

. Dotted – very short dash with spacing equal to

or greater than dash itself

. Dashed – displayed by generating an interdash spacing

Pixel count for the span and interspan length is specified

by the mask . Ex. 1111000111110001111

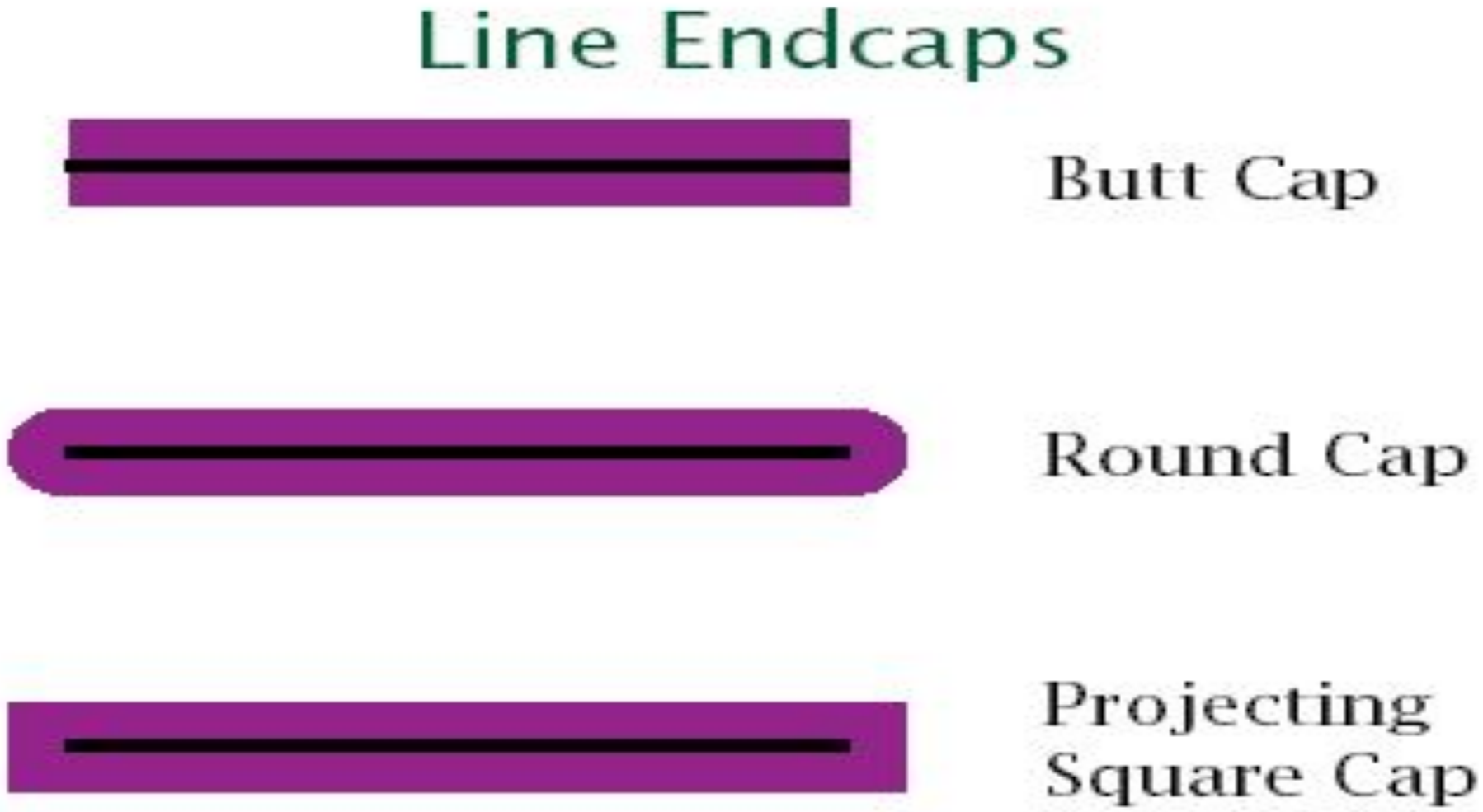
Note : Fixed pixel with dashes can produce unequal length dashes. It depend on line orientation. So, need to adjust the number of plotted pixels for different slopes.

Line Attribute

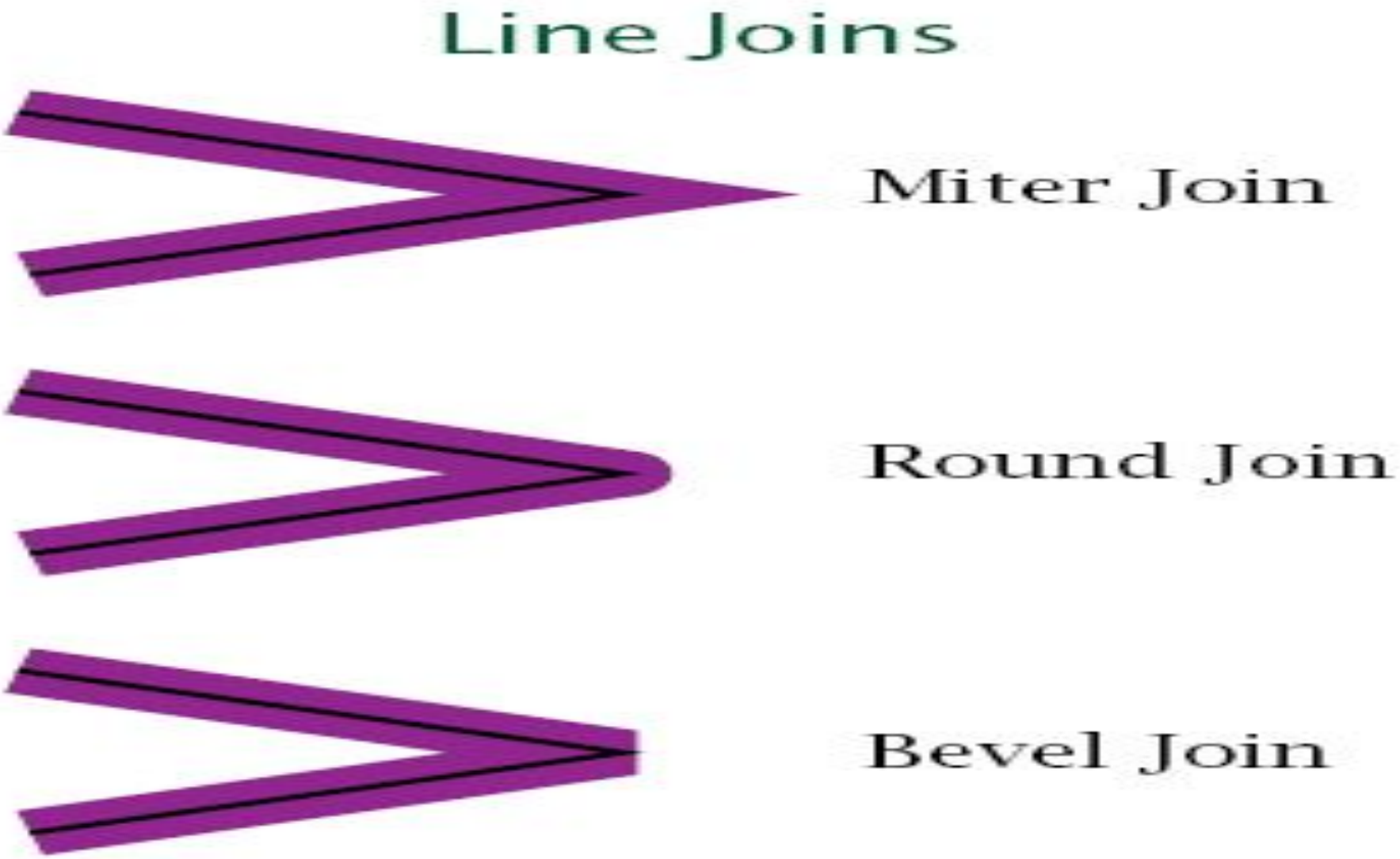
Width

- Specify in pixels and proportion of a standard line width.
- Thicker line can be produced by.
  - Adding extra pixel vertically when $|m| < 1$
  - Adding extra pixel horizontally when $|m| > 1$
- Issues:
  - Line have different thickness on the slope
    - Problem with
      - End of the line
      - Joining the two lines (polygon)

Attributes of Output Primitives



Attributes of Output Primitives



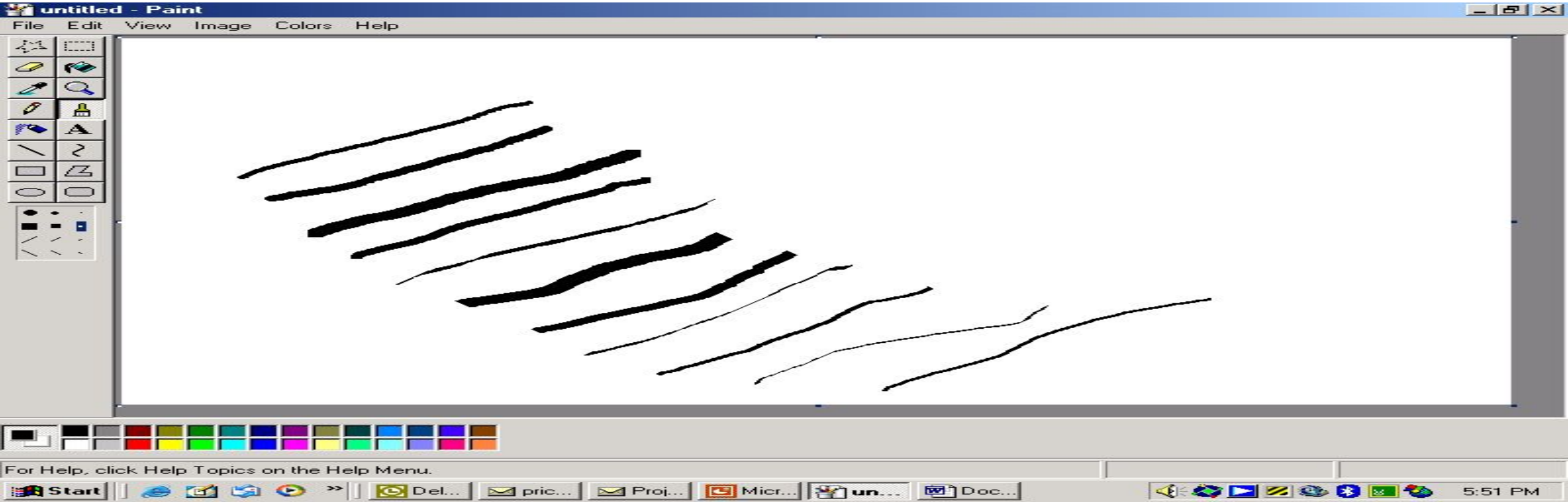
Line Attribute

Pen and Brush

. The selected “pen” or “brush” determine the way a line will be drawn.

. Pens and brushes have size, shape, color and pattern attribute.

. Pixel mask is applied in both of them.

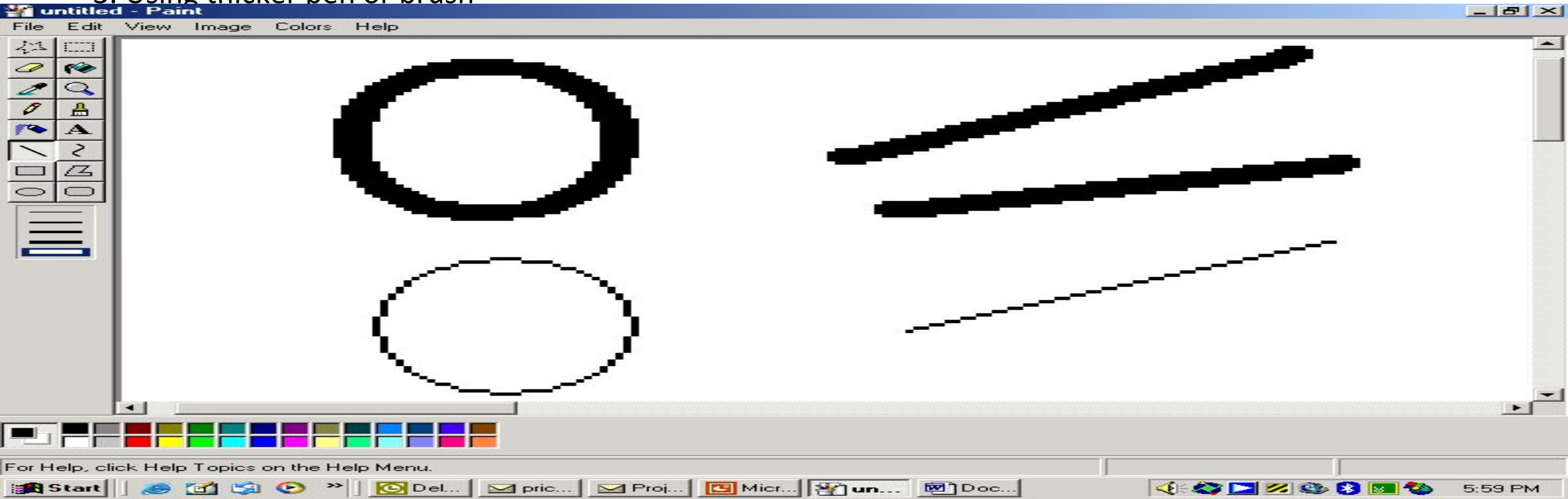


Curve Attribute

Similar to line : type + width

Thicker curves can be produced by:

1. Plotting additional pixel
2. Filling the space between two concentric circles.
3. Using thicker pen or brush



Attributes for characters/strings

- style
- font (typeface)
- color
- size (width/height)
- orientation
- path
- spacing
- alignment

Text attributes

Height 1

Height 2

Height 3

FIGURE 4–36 Text strings displayed with different character-height settings and a constant width-to-height ratio.

width 0.5

width 1.0

width 2.0

FIGURE 4–37 Text strings displayed with varying sizes for the character widths and a xed height.

Spacing 0.0

Spacing 0.5

Spacing 1.0

FIGURE 4–38 Text strings displayed with different character-spacing values.

Text attributes, cont'd

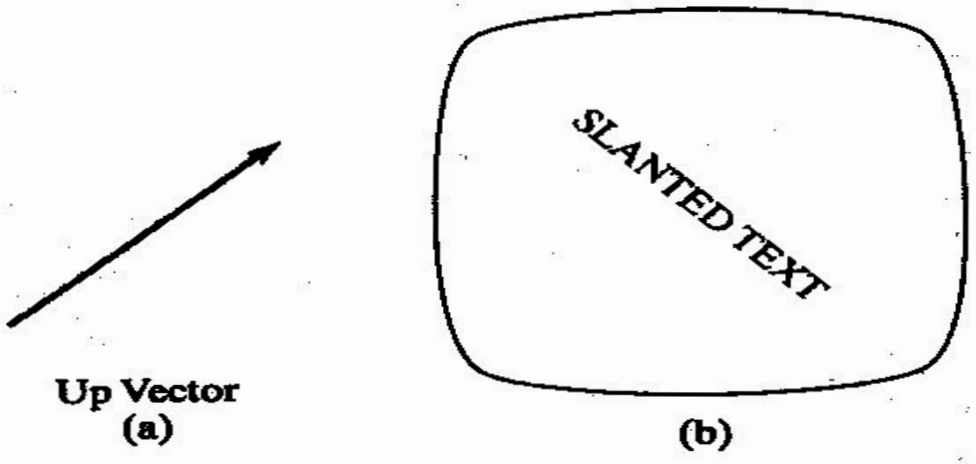


FIGURE 4-39 Direction of the up vector (a) controls the orientation of displayed text (b).

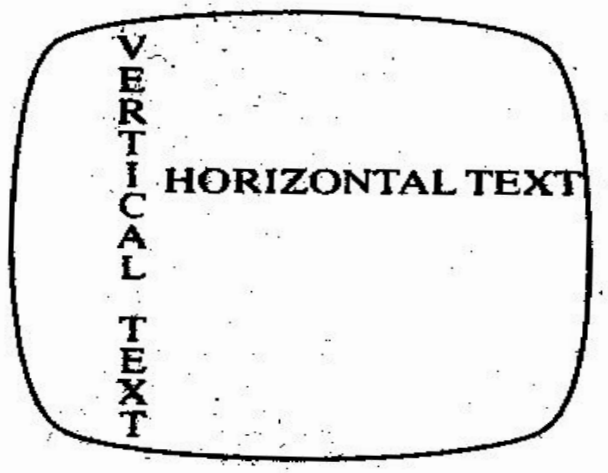


FIGURE 4-40 Text-path attributes can be set to produce horizontal or vertical arrangements of character strings.

Text attributes, cont'd

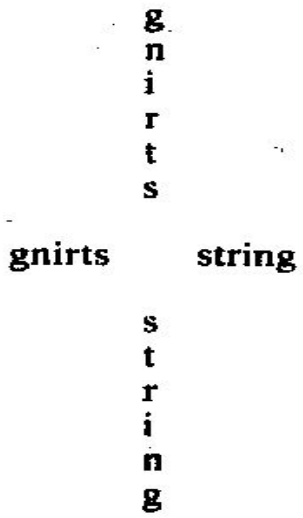


FIGURE 4–41 A text string displayed with the four text-path options: left, right, up, and down.

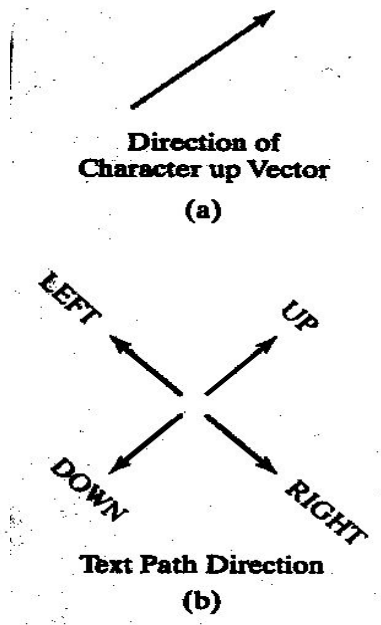


FIGURE 4–42 An up-vector specification (a) and associated directions for the text path (b).

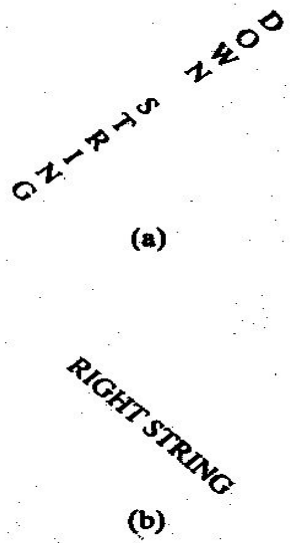


FIGURE 4–43 The 45° up vector in Fig. 4-42 produces the display (a) for a *down* path and the display (b) for a *right* path.

Text attributes, cont'd

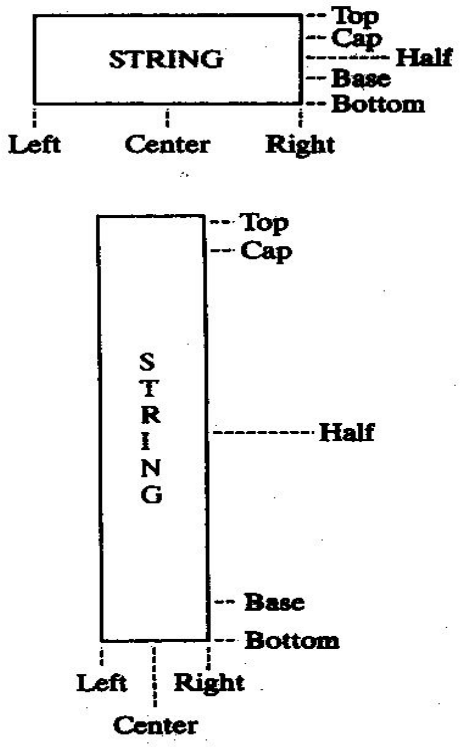


FIGURE 4–44 Character alignments for horizontal and vertical strings.

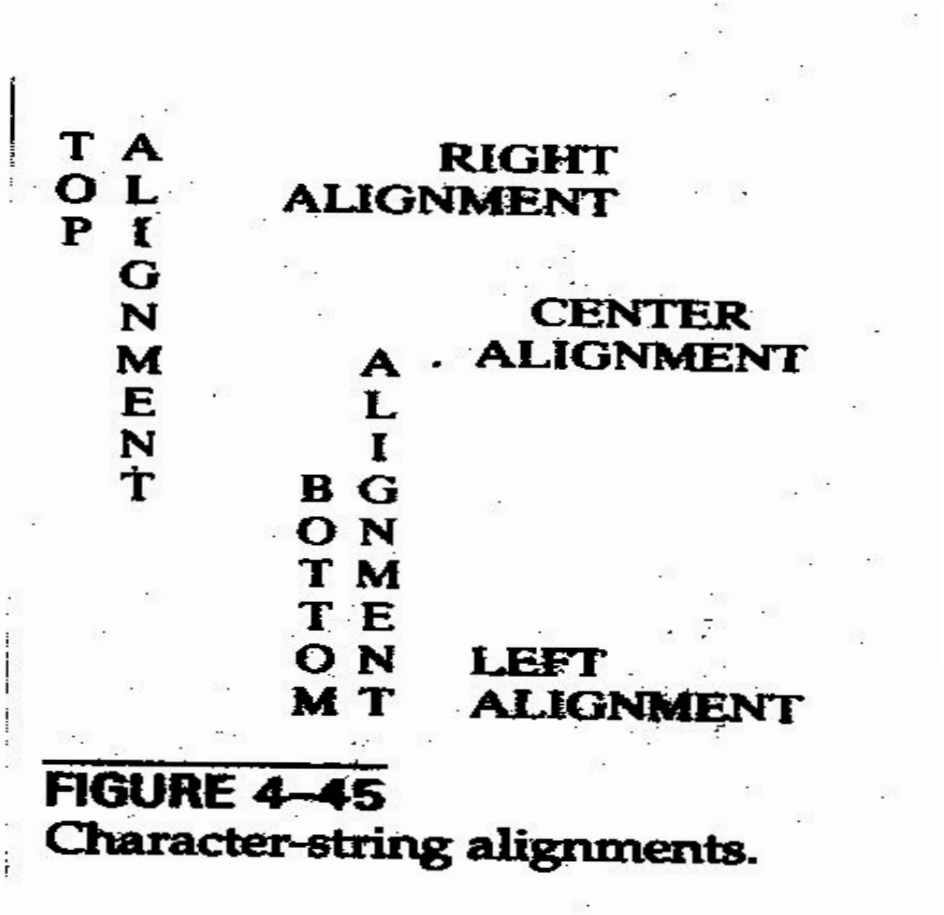


FIGURE 4–45
Character-string alignments.

Color as attribute

Each color has a numerical value, or intensity, based on some color model.

A color model typically consists of three primary colors, in the case of displays Red, Green and Blue (RGB)

For each primary color an intensity can be given, either 0-255 (integer) or 0-1 (float) yielding the final color

256 different levels of each primary color means 3x8=24 bits of information to store

Color representations

Two different ways of storing a color value:

1) a direct color value storage/pixel
2) indirectly via a color look-up table index/pixel (typically 256 or 512 different colors in the table)

COLOR AND GRAY SCALE LEVEL

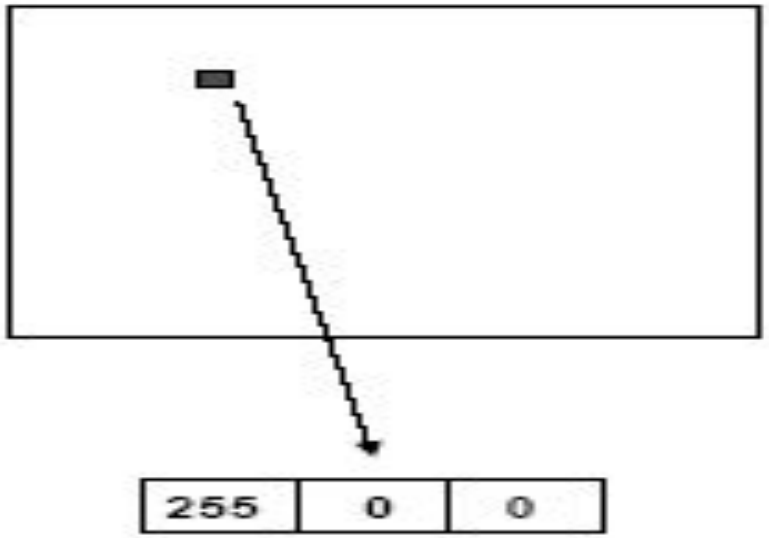
Color

- Colors are represented by colors codes which are positive integers.
- Color information is stored in frame buffer or in separate table and use pixel values as index to the color table.
- Two ways to store color information :
  1. Direct
  2. Indirect

COLOR

Direct

Full-color (RGB) displays

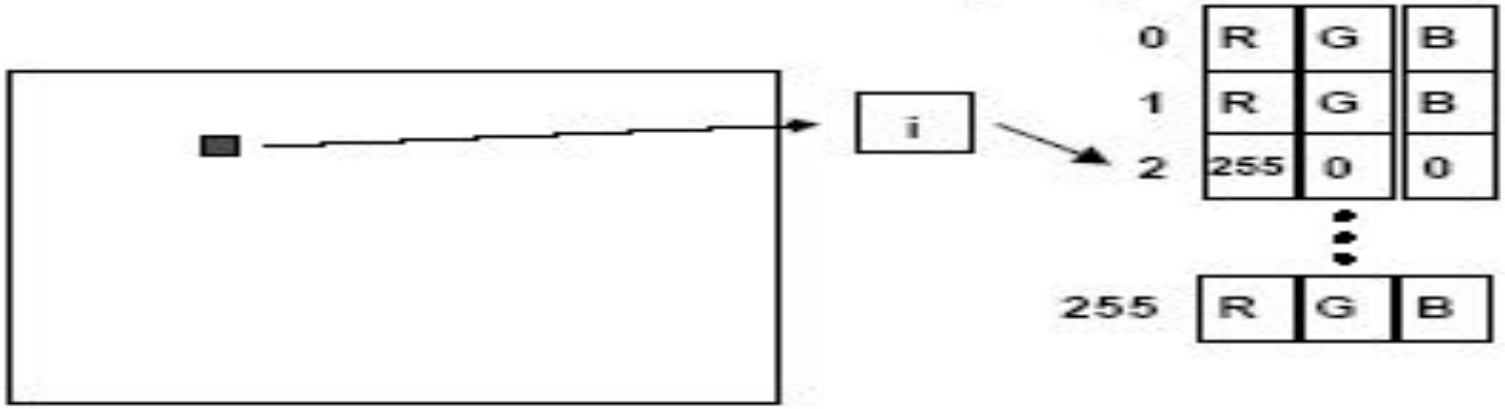


- For 24 bit color:
  - store 8 bits each of red, green, and blue per pixel.
  - E.g. (255,0,0) is pure red, and (255, 255, 255) is white.
  - Yields 2^24 = 16 million colors.
- For 15 bit color:
  - 5 bits red + 5 green + 5 blue
- The video hardware uses the values to drive the R,G, and B guns.
- You can mix different levels of R, G, and B to get any color you want

COLOR

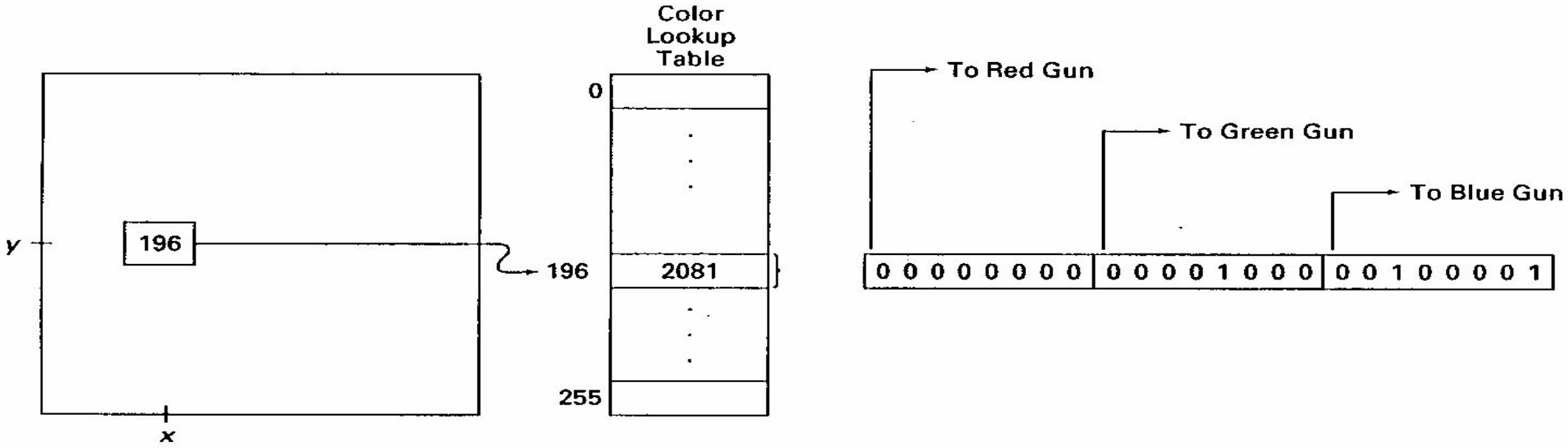
Indirect

Color maps (LUT's)



- A single number (e.g. 8 bits) stored at each pixel.
- Used as an *index* into an array of RGB triples.
- With 8 bits per pixel, you get 256 colors of your choice
- Simple things to fill up color-maps with:
  - A grey ramp (for grey scale pictures)
  - A bunch of random colors (for color drawings.)
  - A very poor representation of full color

Color Look-up Table



COLOR

Exercise:

What is the size of frame buffer required for the following cases:

Case 1 (Direct): Resolution of 1024 * 1024 with 24 bits per pixel

Case 2 (Indirect): Same resolution with 8 bit per pixel that indexed out of 16 million available colors

Conclusion

CLUT is good for storage but cant give a very high resolution picture.